

SZAKDOLGOZAT



MISKOLCI EGYETEM

SPAM/HAM üzenetek osztályozása mélytanulási módszerekkel

Készítette:

Ferencsik Róbert

Programtervező informatikus

Témavezető:

Dr. Baksáné Dr. Varga Erika

MISKOLC, 2026

SZAKDOLGOZAT FELADAT

Ferencsik Róbert (BQLOTW) BSc programtervező informatikus jelölt részére.

A szakdolgozat tárgyköre: természetes nyelvfeldolgozás, szövegosztályozás, mélytanulás

A szakdolgozat címe: SPAM/HAM üzenetek osztályozása mélytanulási módszerekkel

A feladat részletezése:

A hallgató feladata egy SPAM/HAM szövegosztályozó rendszer megtervezése és megvalósítása mélytanulási módszerekkel. A dolgozatnak tartalmaznia kell:

1. A szövegosztályozás elméleti hátterének bemutatását. A hagyományos és mélytanulási módszerek összevetését és a szekvenciális modellek szerepét.
2. A felhasznált adathalmaz részletes elemzését.
3. Az előfeldolgozási folyamat kidolgozását és implementálását.
4. Egy kétirányú LSTM (BiLSTM) modell megtervezését és megvalósítását.
5. Az elvégzett kísérletek és a kapott eredmények bemutatását.
6. A módszer korlátainak és továbbfejlesztési lehetőségeinek áttekintését.

Témavezető(k): Dr. Baksáné Dr. Varga Erika, egyetemi docens
Konzulens(ek):

A feladat kiadásának ideje: 2025. szeptember 9.

.....
szakfelelős

EREDETISÉGI NYILATKOZAT

Alulírott **Ferencsik Róbert**; Neptun-kód: BQL0TW a Miskolci Egyetem Gépészmérnöki és Informatikai Karának végzős Programtervező informatikus szakos hallgatója ezennel büntetőjogi és fegyelmi felelősségem tudatában nyilatkozom és aláírásommal igazolom, hogy *SPAM/HAM üzenetek osztályozása mélytanulási módszerekkel* című szakdolgozatom saját, önálló munkám; az abban hivatkozott szakirodalom felhasználása a forráskezelés szabályai szerint történt.

Tudomásul veszem, hogy szakdolgozat esetén plágiumnak számít:

- szószerinti idézet közlése idézőjel és hivatkozás megjelölése nélkül;
- tartalmi idézet hivatkozás megjelölése nélkül;
- más publikált gondolatainak saját gondolatként való feltüntetése.

Alulírott kijelentem, hogy a plágium fogalmát megismertem, és tudomásul veszem, hogy plágium esetén szakdolgozatom visszautasításra kerül.

Miskolc, év hó nap

.....
Hallgató

- | | |
|-------|---------------|
| | |
| dátum | témavezető(k) |

- | | |
|------------------------------|-----------------------------|
| témavezető (dátum, aláírás): | konzulens (dátum, aláírás): |
| | |
| | |
| | |

- | | |
|-------|---------------|
| | |
| dátum | témavezető(k) |

- | | |
|-------|---------------|
| | |
| dátum | témavezető(k) |

- dátum szakfelelős

- Miskolc,
a Záróvizsga Bizottság Elnöke

Tartalomjegyzék

1. Bevezetés	1
2. Szövegosztályozási feladat és módszerek	3
2.1. Természetes nyelvfeldolgozás	3
2.2. Szövegosztályozás	3
2.3. A szövegosztályozás folyamata	4
2.4. Áttérés a hagyományos módszerekről a mélytanulási módszerekre . . .	4
2.5. Felhasznált módszerek	6
2.5.1. Előfeldolgozás	6
2.5.2. Modell	8
3. Előfeldolgozás	11
3.1. Bevezetés a fejezet felépítéséhez	11
3.2. Adathalmaz	11
3.2.1. Adathalmaz eredete	11
3.2.2. Adathalmaz szerkezete és értékei	12
3.3. Előfeldolgozás tervezése	13
3.3.1. Előfeldolgozás szükségessége	13
3.3.2. Előfeldolgozási folyamat	14
3.3.3. Előfeldolgozás implementálása	14
3.3.4. Feltáró adatelemzés, adattisztítás és ellenőrzés	15
3.3.5. Tokenizálás és validálása	17
4. A klasszifikációs modell és tanítási folyamat	19
4.1. Modelltervezés	19
4.1.1. Modell architektúra	19
4.1.2. Hiperparaméter-tér és optimalizálási stratégia	20
4.2. Implementáció	22
4.2.1. Állomány és konfigurációk kezelése	23
4.2.2. Modell, hiperparaméterek és tanulási görbe	23
5. Kísérletek és eredmények	25
5.1. Hiperparaméter-optimalizálás és tanulási görbe	25
5.1.1. Kísérleti környezet	25
5.1.2. A három fázis keresési tere	26
5.1.3. Első fázis: kezdetleges keresési tér	26
5.1.4. Második fázis: szűkített tér	28
5.1.5. Harmadik fázis: Általánosító képesség	31

6. Összefoglalás és továbbhaladási irány	34
6.1. Összefoglalás (magyar)	34
6.2. Summary (english)	35
Források	37

1. fejezet

Bevezetés

Az információs technológiák rohamos fejlődése alapjaiban alakította át a kiberbiztonságot. Az új technológiák megjelenésével ugyan jelentősen bővült a védekezés eszköztára, ez azonban korántsem jelenti a fenyegetések megszűnését vagy csökkenését. A digitális térben zajló „küzdelem” inkább egy folyamatosan változó, dinamikus folyamatként írható le, ahol a támadási és védekezési módszerek egymással párhuzamosan fejlődnek. Az internet széles körű elterjedése áthelyezte a színterét, napjainkban pedig a mesterséges intelligencia biztosít új eszközöket mind a támadók, mind a védekezők számára.

A köztudatban élő kép, miszerint a kiberbiztonság valós idejű „programozói párbajként” értelmezhető, nem tükrözi a valóság összetettségét. A gyakorlatban a hangsúly sokkal inkább a már működő rendszerekben található sérülékenységek feltárásán és kezelésén van, hogy a támadók ne is férjenek hozzá sérülékeny felülethez. Ez a védelem nem csupán a forráskód minőségétől függ, hanem számos egyéb tényező együttes hatásának eredménye. Ilyen tényező például a titkosítás, a megfelelő jogosultságkezelés, a helyes konfigurációk, és az emberi figyelem és tudás.

Az egyik leggyakoribb támadási forma az **adathalászat** (phishing). Az ilyen típusú támadások során a támadók megtévesztő, gyakran hitelesnek tűnő üzenetek segítségével próbálnak érzékeny adatokat megszerezni a felhasználóktól. Az adathalász e-mailek különösen veszélyesek, mivel kihasználják a felhasználók figyelmetlenségét, illetve döntéshozatali korlátait. Mindez rámutat arra, hogy a kizárólag felhasználói döntésekre építő védekezés nem tekinthető elegendőnek.

Ebből következően a modern kiberbiztonsági megoldások egyik kulcseleme az automatizált védelem alkalmazása, mely a valóságban a természetes nyelv terén, illetve tapasztalat alapján egyéb területeken is inkább nevezhető félautomatának. Az e-mail szolgáltatók fejlett szűrőrendszereket használnak annak érdekében, hogy a nem kívánt vagy kártékony üzeneteket még a felhasználóhoz való eljutás előtt azonosítsák és elkülönítsék.

Ezt a folyamatot klasszifikációnak, osztályba sorolásnak tekintjük. Gépi módszerek számos fajtája alkalmazható, és ugyancsak sokféle adat felhasználásával lehet döntést hozni egy e-mail kéretlen voltáról. A szöveg alapú megközelítések mellett a metaadatokból, hálózati jellemzőkből és viselkedési mintákból automata vagy félautomata módon információt kinyerő modellek és ezek konkrét megvalósítása végtelen számú megoldást nyújt a feladatra.

A beérkező üzenetek nagy mennyisége és folyamatosan növekvő változatossága miatt a modern rendszerek jellemzően **gépi tanulási** (Machine Learning) módszerekre épülnek, amelyek képesek alkalmazkodni az újonnan megjelenő mintázatokhoz, vala-

mint a dinamikusan változó és egyre kifinomultabb támadási technikákhoz. E megközelítések közül különösen kiemelkednek a **mélytanulási** (Deep Learning, DL) módszerek, amelyek a hagyományos gépi tanulási eljárásokhoz képest számos jelentős előnnyel rendelkeznek.

A mélytanulási modellek egyik fontos tulajdonsága, hogy jelentősen csökkentik a kézi **jellemzőtervezés** (Feature Engineering) szükségességét, amely a klasszikus módszerek esetében gyakran időigényes, szakértelmet igénylő folyamat. Ezzel szemben a mély neurális hálózatok képesek közvetlenül a nyers adatokból automatikusan kinyerni a releváns jellemzőket és mintázatokat, ezáltal a tanulási folyamat sokkal inkább adatvezéreltté válik. Ennek eredményeként a modell nemcsak hatékonyabban képes kezelni a nagy mennyiségű és komplex adatokat, hanem jobb általánosító képességet is mutathat új, korábban nem látott példák esetén. A számítási költség növekedése emelhető ki hátránnyként a mélytanulási módszerek kapcsán. Az egyik megközelítés a **kéretlen levél** (spam) és **valódi levél** (ham) kategóriák osztályba sorolására a természetes nyelvű szekvenciák vizsgálata. Ezen dolgozat ezt veszi alapul, és vizsgálja mélytanulási módszerekkel. A természetes nyelvi szekvenciák feldolgozására különösen alkalmasak a **hosszú-rövid távú memória** (Long Short-Term Memory, LSTM) architektúrán alapuló visszacsatolt neurális hálózatok, amelyek a nyelvben előforduló statisztikai mintázatokat tanulják meg. A szöveges e-maileket adat sorozatként kezelik, ahol az egyes szavak és nyelvi mintázatok jelentése nagyban függ az őket megelőző kontextustól. A neurális hálózat a korábban látott szókapcsolatok és statisztikai összefüggések alapján tanulja meg felismerni azokat a mintákat, amelyek jellemzőek lehetnek a kéretlen vagy a valódi levelekre. Az osztályozási döntés során nem kizárólag egyes kulcsszavak jelenlétét vizsgálja, hanem azok sorrendjét, gyakoriságát és környezetét is figyelembe veszi. Ennek köszönhetően a modell képes összetettebb nyelvi és stilisztikai mintázatok felismerésére is, amelyek hozzájárulnak az e-mailek pontosabb osztályozásához.

A dolgozat központi témája a hosszú-rövid távú memóriahálózatokra épülő szöveg-osztályozás gyakorlati vizsgálata. A munka során az LSTM-alapú megközelítés implementációja, tanítása és kiértékelése kerül bemutatásra.

A modell megfelelő kialakítása és paraméterezése mellett az előfeldolgozás lépések részletes megvalósítása és értékelése is jelentős, mivel jelentősen befolyásolják a tanítás hatékonyságát és a végső teljesítményt.

A vizsgálat fontos célja annak feltárása, hogy a tanítóadatok mennyisége milyen mértékben befolyásolja a modell teljesítményét és általánosító képességét. Ennek érdekében a dolgozat elemzi, hogyan változnak a teljesítménymutatók a növekvő tanítóhalmaz függvényében.

2. fejezet

Szövegosztályozási feladat és módszerek

Ebben a fejezetben a kontextus megteremtése érdekében a természetes nyelvfeldolgozás és a szövegosztályozás alapfogalmai kerülnek bevezetésre. Ezt követően a szövegosztályozás folyamatáról esik szó. Végül a felhasznált módszerek részletezése következik.

2.1. Természetes nyelvfeldolgozás

A természetes nyelvfeldolgozás olyan tudományág, amely lehetővé teszi a számítógépek és digitális rendszerek számára az emberi nyelv értelmezését, megértését és generálását. Integrálja a számítógépes nyelvészetet, a szabályalapú megközelítéseket, a statisztikai modellezést, valamint a korszerű mélytanulási technikákat [14].

A természetes nyelvfeldolgozás területén folyó kutatások döntő szerepet játszottak a modern nyelvtechnológiai rendszerek fejlődésében, elősegítve a szöveges és beszélt nyelv értelmezésére, generálására és összetett feladatok végrehajtására képes megoldások kialakulását [14].

2.2. Szövegosztályozás

A szövegosztályozás egy olyan folyamat, amely során egy dokumentumot automatikusan egy vagy több kategóriába sorolnak. Létezik felügyelt és nem felügyelt változata.

A **felügyelt gépi tanulás** (Supervised Learning) módszer alapja a dokumentum belső jellemzőinek elemzése, majd egy előre címkézett adathalmaz segítségével modell létrehozása a dokumentum és a kategória közötti kapcsolatok feltérképezésére [17].

A szövegosztályozás jelentősége az **adatrobbanás korszakával** (Era of Big Data) tovább nőtt, hiszen lehetővé teszi a strukturálatlan szöveges adatok hatékony szervezését és feldolgozását. Számos tanulmány rámutat arra, hogy a szövegosztályozás nem csupán egy a sok természetes nyelvfeldolgozás feladat közül, hanem a terület alapvető építőeleme. [8] megállapítja, hogy „a szövegbesorolás a természetes nyelvfeldolgozás legfontosabb és legalapvetőbb feladata” (saját fordítás).

2.3. A szövegosztályozás folyamata

A folyamat tipikusan több egymást követő lépésből áll, melyet a 2.1. ábra szemléltet. Kezdve a **szöveg előfeldolgozásával** (preprocessing), amely magában foglalja a tokenizálást, a stop-szavak eltávolítását, a szöveg kisbetűsítését, valamint a szótövezést vagy a lemmatizálást. Az előfeldolgozás célja egy egységes, zajtól megtisztított és könnyen feldolgozható szövegállomány előállítása, amely stabil alapot biztosít a további lépésekhez. Ezt követi a **jellemzőkinyerés** (feature extraction), amelynek során a dokumentumok numerikus reprezentációra alakíthatók. Ez a reprezentáció teszi lehetővé, hogy a különböző gépi tanulási algoritmusok a szöveg struktúráját és tartalmi jellemzőit kvantitatív módon értelmezzék.



2.1. ábra: A szövegosztályozás tipikus folyamata gépi tanulási módszerekkel. *Forrás:* [8].

Az osztályozási modell létrehozása előtt általában sor kerül a tanított modell **kiértékelésére** (evaluation), amely során különböző teljesítménymutatók – például precizitás, visszahívás, F1-mérték – segítségével vizsgálható, hogy a modell milyen mértékben képes a különböző kategóriák helyes elkülönítésére. A kiértékelés elengedhetetlen lépés annak meghatározásához, hogy a modell alkalmas-e éles környezetben történő alkalmazásra, illetve hogy szükség van-e további finomhangolásra vagy alternatív jellemzők használatára.

A folyamat végén az **osztályozó** (classifier) a dokumentumot a megfelelő **címkéhez** (label) rendeli, amely a későbbi elemzési feladatok kiindulópontja lehet. A szövegosztályozás eredményei többféle további feldolgozást támogatnak, például **érzelmi állapotok feltárását** (sentiment analysis), **témaazonosítást** (topic analysis), spam- vagy anomáliaészlelést, valamint nagy szövegtörzsek strukturálását és információ-visszakeresési rendszerek hatékonyságának javítását. A címkézett adatok ezen túlmenően alapot biztosítanak prediktív modellekhez, ajánlórendszerekhez, illetve bármely olyan automatizált folyamat számára, ahol nagy mennyiségű szöveges adat gyors és megbízható kategorizálása szükséges.

2.4. Áttérés a hagyományos módszerekről a mélytanulási módszerekre

[8] szerint „az 1960-as évektől a 2010-es évekig a hagyományos szövegbesorolási módszerek domináltak” (saját fordítás), és a mélytanulási módszerekre való áttérés 2010-ben kezdődött. A hagyományos módszerek statisztikán alapulnak, és a korábbi szabályalapú módszerekhez képest nagyobb pontosságot és stabilitást mutattak. Az adatrobbanás korában azonban a manuális jellemzőkinyerés költséges és időigényes, a statisztikai megközelítések pedig figyelmen kívül hagyják a szöveges adatok természetes szekvenciális szerkezetét vagy kontextusbeli információit.

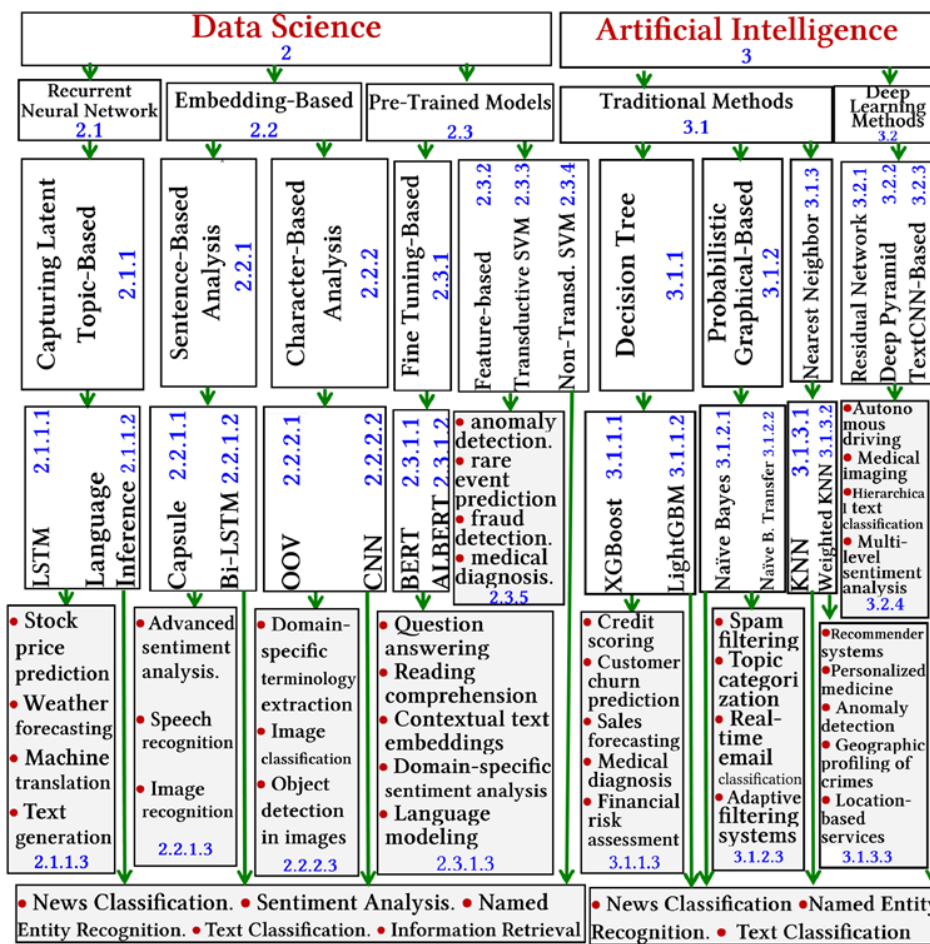
A hagyományos megközelítésekkel szemben a mélytanulási módszerek kiküszöbölik a kézzel készített szabályok és kézi jellemzőkinyerés szükségességét, mivel képesek az adatokból automatikusan, szemantikailag gazdag és kontextusérzékeny reprezentációkat tanulni. Ez a tulajdonság lehetővé teszi a **mély neurális hálózatok** (Deep Neural Network, DNN) számára, hogy olyan összetett nyelvi mintázatokat, hosszú távú függőségeket és finom szemantikai különbségeket is megragadjanak, amelyeket a hagyományos technikák jellemzően nem képesek megfelelő pontossággal modellezni. Ennek következtében a szövegosztályozás területén végzett jelenkori kutatások döntő része mély neurális hálózatokra épül, amelyek adatvezérelt architektúrákként automatikusan képesek hierarchikus jellemzők kinyerésére, és számos alkalmazási területen a legkorszerűbb teljesítményt biztosítják. Ugyanakkor a mélytanulási modellek hatékony működéséhez általában jelentős számítási kapacitásra és nagy méretű, annotált tanítóadatokra van szükség. Emiatt a **hagyományos gépi tanulási** (Traditional Machine Learning) módszerek iránti érdeklődés elsősorban azokban a scenáriókban maradt fenn, ahol a számítási erőforrások korlátozottak, az adatmennyiség csekély, vagy valós idejű feldolgozási követelmények teszik szükségessé a könnyebb, kompaktabb modellek alkalmazását.

A szövegosztályozási technikák két nagy csoportba sorolhatók: **adattudomány** (Data Science) és **mesterséges intelligencia** (Artificial Intelligence) [15].

Itt érdemes megemlíteni, hogy ez a két osztály nem diszjunkt, hanem az adattudomány egy szélesebb terület, amelynek egyik eszköze lehet a mesterséges intelligencia. Ezt a következtetést definíciójukból le lehet vonni.

Az adattudomány: „Olyan tudományág, amely tudományos módszereket, folyamatokat és rendszereket alkalmaz az adatokból származó ismeretek és betekintések kinyerésére” [16] (saját fordítás).

A mesterséges intelligencia egyik definíciója: „Egy technológia, amely felruházza a számítógépeket és gépeket az emberi tanulás, megértés, problémamegoldás, döntéshozatal, kreativitás és önállóság utánzásának képességével” [4] (saját fordítás).

2.2. ábra: A szövegosztályozási módszerek taxonómiája. *Forrás:* [15].

2.5. Felhasznált módszerek

A felhasznált módszerek két csoportban kerülnek bemutatásra. Az első csoport az adathalmaz feldolgozására vonatkozik, míg a második a hosszú-rövid távú memória neurális háló paramétereire és konkrét típusára. Az egyszerűbb és közismertebb módszerek nem kerülnek bemutatásra, az implementáció részletezésénél felsorolásra kerülnek.

2.5.1. Előfeldolgozás

Tukey interkvartilis tartomány

Az adathalmazt tekintve az üzenetek hossza egy nagyon eltérő tartományon oszlik el. A rövid, pár szavas üzenetektől egészen a hosszú üzenetváltásokig, melyek akár százazres nagyságrendbe is lépnek karakterszám tekintetében. A döntés meghozatala, hogy melyik üzenet túl rövid, és melyik túl hosszú, statisztikai alapon fekszik. Méghozzá **Tukey interkvartilis módszerén** (Tukey's Interquartile Range Method, Tukey's IQR-method).

Az **interkvartilis tartomány** (interquartile range, IQR) egy statisztikai mutató mely információt ad az adatok eloszlásáról, azonban egyike azon módszereknek melyet nem befolyásol jelentősen az adatok nem normális jellege, esetünkben az eloszlás **ferdesége** (skewness). A harmadik (Q3) és első kvartilis (Q1) különbsége adja értékét

[12].

$$IQR = Q_3 - Q_1$$

A Tukey-módszer alapján a kiugró értékek az alábbi határok segítségével határozhatók meg:

$$\text{alsó határ} = Q_1 - 1.5 \cdot IQR$$

$$\text{felső határ} = Q_3 + 1.5 \cdot IQR$$

Entitások helyettesítése

Az **entitások helyettesítése** (Entity Masking) számos természetesnyelv-feldolgozási feladat egyik fontos előfeldolgozási lépése. Az eljárás során a **megnevezett entitások** (Named Entities), például személynevek, telefonszámok, e-mail címek, dátumok vagy helynevek helyére előre definiált helyettesítő tokenek kerülnek, miközben az adott elem szemantikai szerepe továbbra is megmarad a szövegben. A módszer célja feladattól függően eltérő lehet, a jelen dolgozat esetében azonban elsősorban a szókészlet méretének csökkentése volt a cél. Az entitások helyettesítésével a modell kevésbé a konkrét karakterláncokat, sokkal inkább azok kontextuális szerepét és a szövegben megjelenő mintázatokat tanulja meg.

Az entitásmaszkolás további természetesnyelv-feldolgozási feladatok során is széles körben alkalmazott technika. Gyakori felhasználási területe az adatvédelem, ahol szükséges a személyes vagy érzékeny adatok eltávolítása a modellek tanítása előtt. Emellett jelentős szerepet játszik a szókészlet csökkentésében és a szöveges adatok normalizálásában is, mivel az azonos típusú entitások egységes reprezentációt kapnak. Ez javíthatja a modellek generalizációs képességét, mivel a hálózatok kevésbé a konkrét értékeket memorizálják, hanem az azokhoz kapcsolódó nyelvi mintázatokat és kontextusfüggéseket sajátítják el. A generatív és előtanított nyelvi modellek esetében a maszkolás az előtanítás egyik fontos eleme is lehet, ahol a modellek feladata a maszkolt tokenek predikciója. Az ilyen jellegű tanítás lehetővé teszi, hogy a modellek mélyebb szemantikai és szintaktikai reprezentációkat tanuljanak a nyelvi adatokból.

SentencePiece tokenizálás

A Kudo és Richardson által 2018-ban bemutatott SentencePiece egy nyelvfüggetlen résszavas tokenizáló rendszer mely képes nyers, zajos szövegekből szókészlet-tanulásra.

A működésének alapja egy statisztikai szegmentáló algoritmus alkalmazása a szöveg apróbb egységekre bontására, előfeldolgozás (tokenizálás, kisbetűsítés stb.) szükségése nélkül. A teljes betanítás során a beviteli szövegben található karakterláncokat közvetlenül a modell megtanulja, nem kell előre tokenizált szavakkal dolgozni. Ennek eredményeként a SentencePiece nyelvfüggetlen feldolgozást tesz lehetővé: ugyanazon algoritmus alkalmazható bármilyen nyelven, akár szóközzel tagolt (angol), akár összefüggő (japán, kínai) szövegeken is, és egyetlen, előre rögzített méretű szótárra építve hozza létre a tokeneket [7].

A módszer további előnye a veszteségmentes tokenizáció, mellyel a különleges karakterek, sőt még a szóközők információja is megmarad. Így visszaállítható az eredeti

szöveg és bármikor biztonságosan reprodukálható a tokenizációs folyamat. Illetve alkalmazása jelentősen csökkenti a hagyományos jellemzőtervezés szükségességét, és elősegíti a modellek számára a releváns mintázatok tanulását [7].

E-mail (spam/ham) osztályozáskor a SentencePiece különösen előnyös a szövegek zajossága és változatossága miatt. A spam üzenetek gyakran tartalmaznak elgépeléseket, felesleges karaktereket, melyeket szótári módszerekkel nehéz és költséges kezelni. Mindez robusztusabbá teszi a rendszer működését, és a félreírt, felcserélt vagy egymáshoz fűzött szóalakokat sem kezeli „ismeretlenként”. A résszavas bemenet ráadásul a modern neurális hálózatok számára is ideális.

Az irodalomban nem sűrűn fordul elő, ha egyáltalán előfordul, a SentencePiece használata hosszú-rövid távú memória modellek esetében. Ennek oka a két módszer megjelenésének időpontja és aktív használata lehet. A hosszú-rövid távú memória modellek korszakában, körülbelül 2015-től 2019-ig még a Word2Vec, GloVe és a sima szó-tokenizálás volt a jellemző, míg a SentencePiece elsősorban a Transformer, BERT, többnyelvű módszerekkel lett népszerű.

2.5.2. Modell

Kétirányú hosszú-rövid távú memória neurális háló

A **kétirányú hosszú-rövid távú memória** (Bidirectional Long Short-Term Memory, BiLSTM) neurális hálózat a **rekurrens neurális hálók** (RNN) egy továbbfejlesztett változata. A klasszikus LSTM modellekkel ellentétben, amelyek a szöveget csak egy irányban (balról jobbra) dolgozzák fel, a BiLSTM architektúra két különálló LSTM réteget alkalmaz. Az egyik a bemeneti szekvenciát előre, a másik pedig visszafelé dolgozza fel. Ennek eredményeként a modell minden token esetében hozzáfér mind az előző, mind a következő kontextushoz, ami jelentősen javítja a szöveges reprezentáció minőségét[19].

Szövegklasszifikációs feladatok esetében ez különösen előnyös, mivel a szavak jelentése erősen függ a környező tokenektől, és gyakran a későbbi kontextus is szükséges a helyes értelmezéshez. 2016-ban kimutatták, hogy a kétirányú hosszú-rövid távú memória modellek hatékonyabban képesek modellezni a hosszabb távú összefüggéseket a szövegben mint az egyirányú hosszú-rövid távú memória modellek [9].

További módszerekkel még pontosabb eredményeket lehet elérni, például a **figyelem** (attention) mechanizmusának alkalmazásával, azonban ezekre a módszerekre a jelen dolgozat nem tér ki.

Véletlenszerű hiperparaméter-keresés

A **véletlenszerű hiperparaméter-keresés** (Hyperparameter Optimization with Random Search) egy népszerű hiperparaméter-optimalizálási módszer, amely a **rácskereséssel** (Grid Search) ellentétben nem egy előre meghatározott rácsos elosztást vizsgál, hanem az előre megállapított tartományból véletlenszerűen mintavételez. Bergstra és Bengio szerint a véletlenszerű keresés hatékonyabb, mint a rácskeresés mivel hasonló teljesítményű beállítást talál a modellnek az idő töredéke alatt, és jelentősen kevesebb számítási kapacitást igényel. Ugyanakkora számítási erőforrás mellett a véletlenszerű próbálkozás több különböző kombinációt fed le, így nagyobb eséllyel talál magára a fontos beállításokra. Szóval a véletlenszerű keresés megtartva

a rácskeresés előnyeit, (egyszerű implementálás, egyértelmű párhuzamosítás), magas dimenziójú feladatokban komoly hatékonyság növekedést eredményez [1].

Az adott feladathoz egyszerűsége és gyors implementálhatósága elengedhetetlen volt, mivel adott kérdésekre ad választ a dolgozat, nem egy gyártásba tervezett módszer, így az időköltés volt az első szempont. Ez a módszer pedig megfelelő értékeket ad a hiperparaméterek optimalizálására.

Kiértékelés

A modell kiértékelésére több klasszikus mérőszám áll rendelkezésre, amelyek alkalmasak a szöveghosszú feladatok kiértékelésére. A vizsgálat részeként egy **kumulatív tanítási görbe** (Learning Curve) is alkalmazásra került, mely a tanító adat fokozatos bővítésével mutatja a modell teljesítményének alakulását az F1-pontszám alapján.

Ez a megközelítés lehetővé teszi annak vizsgálatát, hogy a rendelkezésre álló adatmennyiség növekedése hogyan befolyásolja a modell általánosító képességét. A görbe így nem a tanítási folyamat konvergenciáját, hanem az adatmennyiség és a modell F1-pontszáma közötti kapcsolatot szemlélteti. Amennyiben a görbe telítődést mutat, az arra utalhat, hogy a további adatmennyiség már nem eredményez jelentős javulást a modell F1-pontszám értékében.

A klasszifikációs teljesítmény részletesebb elemzésére továbbá rendelkezésre áll az úgynevezett **konfúziós mátrix** (Confusion Matrix), mely a valós és prediktált osztályok közötti összefüggéseket mutatja. A mátrix elemei a következők:

	Prediktált pozitív	Prediktált negatív
Valós pozitív	Igaz Positív	Hamis Negatív
Valós negatív	Hamis Positív	Igaz Negatív

- **Igaz Positív** (True Positive, TP): a modell helyesen spamnek (pozitív osztály) jelöl egy valóban spam üzenetet.
- **Igaz Negatív** (True Negative, TN): a modell helyesen hamnak (negatív osztály) jelöl egy valóban nem spam (ham) üzenetet.
- **Hamis Positív** (False Positive, FP): a modell spamnek jelöl egy valójában ham üzenetet (téves riasztás).
- **Hamis Negatív** (False Negative, FN): a modell hamnak jelöl egy valójában spam üzenetet (el nem kapott spam).

A konfúziós mátrix alapján került kiszámításra a **precizitás** (precision), a **visszahívás** (recall) és az **F1-érték** (F1-score) [5].

A precizitás azt mutatja meg, hogy a pozitívnak prediktált minták közül mennyi volt valóban pozitív, amelyet az 2.1 egyenlet definiál.

$$\text{Precizitás} = \frac{TP}{TP + FP} \quad (2.1)$$

A visszahívás azt fejezi ki, hogy a tényleges pozitív minták hány százalékát sikerült helyesen azonosítani, amelyet az 2.2 egyenlet ír le.

$$\text{Visszahívás} = \frac{TP}{TP + FN} \quad (2.2)$$

Az F1-érték a precizitás és a visszahívás harmonikus átlaga, amely kiegyensúlyozott teljesítménymutatót biztosít, különösen osztály-egyensúlytalanság esetén használatos, vagy abban az esetben ha a precizitás és visszahívás egyformán fontos [5]. Az F1-érték számítását az 2.3 egyenlet mutatja be.

$$F1 = 2 \cdot \frac{\text{Precizitás} \cdot \text{Visszahívás}}{\text{Precizitás} + \text{Visszahívás}} \quad (2.3)$$

3. fejezet

Előfeldolgozás

3.1. Bevezetés a fejezet felépítéséhez

Az előfeldolgozásról szóló fejezetben bemutatásra kerül az adathalmaz rövid történeti háttere, valamint az előfeldolgozási folyamatok. A fejezet ismerteti a végrehajtás lépéseit a nyers, vesszővel tagolt CSV formátumú üzenetekről kezdve egészen a tanítási, validációs és tesztelési adathalmazok előállításáig, beleértve a tokenizáló modell elkészítését is.

Az implementációt ismertető részekben a hangsúly a „miért” és a „hogyan” kérdéseken van, nem pedig a teljes kódbázis részletes bemutatásán. A futtatható kód a verziókezelte forráskódban és a Jupyter notebookokban érhető el, míg a dolgozat első sorban a döntések indoklását és a be- és kimenetek közötti kapcsolat dokumentálását tartalmazza.

3.2. Adathalmaz

3.2.1. Adathalmaz eredete

A felhasznált adathalmaz M. Wiechmann `enron_spam_data` GitHub-oldalán található vesszővel tagolt adatfájl (CSV) formátumban [18].

Az eredeti adathalmazról, mely abban különbözik a leírtak szerint, hogy minden egyes e-mail külön fájlként van tárolva, a *Spam filtering with Naive Bayes – Which Naive Bayes?* tanulmány harmadik fejezetében olvashatunk. A következő bekezdések ezt a művet veszik alapul az adathalmaz általános bemutatására [10].

Az Enron botrány kivizsgálásának keretében közel 150 Enron dolgozó fájljai lettek közzé téve, melyek között jelentős mennyiségű személyes e-mail üzenet is megtalálható. Személyre szabott spam szűrőfejlesztéséhez kiválasztottak 6 Enron dolgozót: `farmer-d`, `kaminski-v`, `kirchen-l`, `williams-w3`, `beck-s` és `lokay-m`, a Bekkerman által szolgáltatott adathalmazból, melyben a levelezések csak ham üzeneteket tartalmaztak [10].

A spam üzeneteket négy különböző forrásból állították össze, melyek sorra: a SpamAssassin corpus, a Honeypot project, Bruce Guenter spam gyűjteménye, és Paliouras G. egyéni spam gyűjteménye. A hat darab ham üzenet gyűjteményből mindegyik párba lett állítva egy spam gyűjteménnyel a három közül, és az arányuk három esetben 1:3-hoz lett, míg a másik három esetben 3:1-hez [10].

A közzétett korpusz készítői az alábbi előfeldolgozási lépéseket is elvégezték:

- az e-mail fiók tulajdonosa által küldött levelek eltávolítása;
- HTML címkék eltávolítása;
- e-mail fejléc eltávolítása;
- nem latin karaktereket alkalmazó spamek eltávolítása.

3.2.2. Adathalmaz szerkezete és értékei

Összesen 33.716 üzenet található az adathalmazban kéretlen és valódi válogatás nélkül. Az adathalmaz öt oszlopban tárolja az adatokat egyes üzenetváltásokról. Ez az öt oszlop sorra az index, a tárgy, az üzenet törzse, a címke és a dátum. Eredeti néven sorra: Message ID, Subject, Message, Spam/Ham és Date. Az első bejegyzés a fejléc információkkal együtt a következő:

```
Message ID,Subject,Message,Spam/Ham,Date
0,christmas tree farm pictures,,ham,1999-12-10
```

A címkék egyensúlyban vannak, csupán egy százalékponttal tér el a két osztály aránya. Számszerűsítve a spam üzenetek száma 17.171 azaz 50.93%, míg a ham üzenetek száma 16.545 azaz 49.07%. Ez fontos információ a modellezés során, mivel így a **torzítás** (bias) könnyebben elkerülhető, és a pontosság kiértékelésekor nem félrevezető.

A tárgy és az üzenet törzse fontos információ, mivel ezek alapján kerül implementálásra az osztályba sorolás, ezek nyers karakteres formátumúak, és tartalmaznak metadatokat is, idő, feladó stb. Zajosak, speciális karaktereket, táblázatokat, linkeket, és az Enron céghez kapcsolódó specifikus nyelvezetet tartalmaznak. Azonban nem tartalmaznak nagybetűt, és a speciális karakterek köré szóközöket illesztettek, melyek elméletben az adatok gépi módszerrel való feldolgozását nehezítik. A címkék formátuma szöveges spam és ham alakban jelenik meg. A dátum oszlopban a bejegyzések alakja éééé-hh-nn formátumot követi.

A hiányzó adatokról, különböző mintázatokról és duplikációk számáról a következő táblázatok adnak információt.

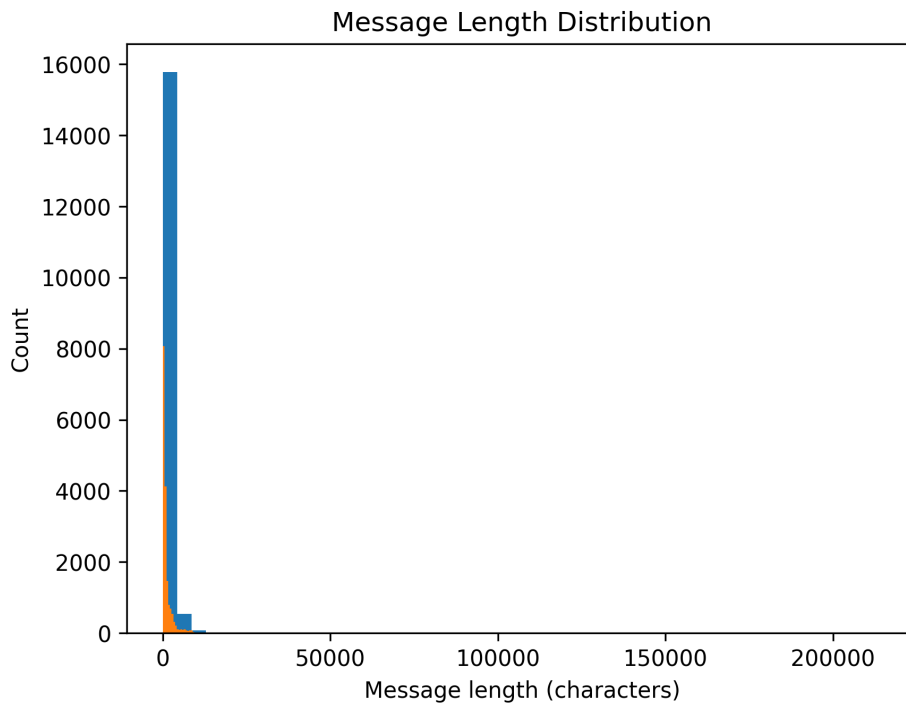
3.1. táblázat: Adathalmaz tisztasági és hiányzó érték statisztikái

Jellemző	Darabszám
Duplikált sorok száma	2953
Hiányzó Message mezők	371
Hiányzó Subject mezők	289
Hiányzó Spam/Ham mezők	0
Hiányzó Date mezők	0

3.2. táblázat: A szöveges adatokban detektált mintázatok és előfordulások

Szövegjellemző	Előfordulás
URL-ek száma	266754
E-mail címek száma	16294
Telefonszámok száma	11992
Számok összesen	435439
Ismétlődő karakterek	79316

Az üzenetek hosszának eloszlása karakter szerint erősen jobbra ferde (A 3.1. ábra), az átlag jóval nagyobb, mint a medián (átlag: 1365, medián: 613), mely azt jelenti, hogy a rövid üzenetek száma magas, azonban van néhány extrém hosszú üzenet, ami felfelé húzza az átlagot. A maximális karakterszámú üzenet 214.422 terjedelmű.



3.1. ábra: Nyers korpusz üzenethossz-eloszlása karakterszámban.

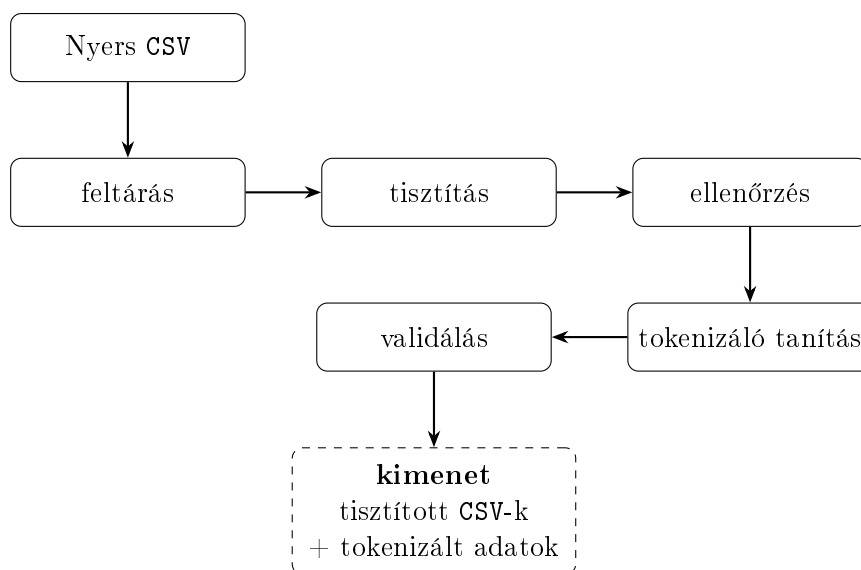
3.3. Előfeldolgozás tervezése

3.3.1. Előfeldolgozás szükségessége

Különböző feladatok és eszközök használatakor különböző kihívások és célok merülnek fel. A hosszú-rövid távú memória modellek esetében két fő célja van az előfeldolgozási lépésnek. Az egyik a zaj csökkentése, hogy a tanítás során a helyes összefüggések kerüljenek feltárára. A másik a szókincs csökkentése mely nagy összetett feladatok esetében is 30.000 maximum 50.000 token.

3.3.2. Előfeldolgozási folyamat

A 3.2. folyamatábra magas szinten mutatja be a lépések közötti kapcsolatot és alapot ad az ábrát követő magyarázatokhoz.



3.2. ábra: A lépések közötti adatáramlás. Az ábra a fájlok mozgását és a felelőségek felosztását hangsúlyozza

Az adatok letöltése után a feltárás vagy **feltáró adatelemzés** (Exploratory Data Analysis, EDA) következett. A bemenetet a nyers adathalmaz képezte, míg a kimenet az előző fejezetben ismertetett részletes információk és az ezek alapján meghozott tisztítási döntések voltak.

A tisztítás során ezen döntések kivitelezése történt meg, amely javarészt adatok eltávolításával, helyettesítésével és átalakításával járt. Az előfeldolgozási lépések végrehajtását egy ellenőrzési szakasz követte, amely során a tisztítás sikeressége és a kívánt tulajdonságok megőrzése került vizsgálatra.

A sikeres tisztítás után került sor a tokenizáló modell tanítására, majd annak validálására. A validálás célja a megfelelő szókészlet ellenőrzése, valamint a klasszifikációs modell helyes paraméterezésének támogatása volt.

3.3.3. Előfeldolgozás implementálása

Az előfeldolgozás reprodukálhatósága érdekében a felhasznált eszközök kerülnek ismertetésre. Maga a folyamat Jupyter-jegyzetfüzetekben került implementálásra. Ezek megnyitásához szükség volt egy megfelelő környezetre, amely lehetett például a Jupyter Notebook, a Google Colab, vagy a Visual Studio Code megfelelő bővítményekkel. A kód Python programozási nyelven készült, és a fejlesztéshez használt verzió a 3.10.0 volt.

A kód futtatásához a következő, pip-pel telepíthető külső csomagok kerültek felhasználásra (zárójelben a dokumentációban feltüntetett verziószámokkal):

- pandas (2.3.3);
- matplotlib (3.10.8);

- scikit-learn (1.7.2);
- sentencepiece (0.2.1);
- numpy (2.2.6).

A jegyzetfüzet-környezet használatához a **jupyter** csomag telepítése volt szükséges. Emellett legalább az **ipykernel** (7.2.0) csomag is kellett.

3.3.4. Feltáró adatelemzés, adattisztítás és ellenőrzés

A fejezet célja az adathalmaz szerkezetének és tulajdonságainak vizsgálata, valamint az előfeldolgozási döntések bemutatása. A feltáró adatelemzésért felelős notebook a `1-eda-1.ipynb`, míg az adattisztítást végző notebook a `2-data-cleaning.ipynb` fájlban található. Az első notebook bemenetét a nyers adathalmaz képezi, kimenete pedig a rétegezetten felosztott adathalmazok és az előfeldolgozási döntések dokumentálása. Az adattisztító notebook bemenetként a felosztott tanító-, validációs- és tesztkészleteket használja, kimeneteként pedig a tisztított adathalmazok állnak elő.

Első lépésként megtörtént az adathalmaz **rétegezett felosztása** (stratified split) tanító (train), validációs (validation) és teszt (test) részhalmazokra. A rétegezett felosztás biztosítja, hogy a spam és ham címkék aránya minden részhalmazban közel azonos maradjon. Ez különösen fontos bináris osztályozási feladatok esetén, mivel az osztályeloszlás torzulása kedvezőtlenül befolyásolhatja a modell tanítását és kiértékelését.

Az adattisztítás során a dátum oszlop eltávolításra került, mivel nem hordoz releváns információt a spam-osztályozási feladat szempontjából. A tárgymező (subject) az üzenet törzséhez került hozzáfűzésre annak érdekében, hogy a modell az e-mail teljes szöveges tartalmát egységes reprezentációként kezelhesse. A hiányzó értékeket és duplikált rekordokat tartalmazó adatsorok száma elhanyagolható volt, ezért ezek eltávolítása nem eredményezett jelentős adatvesztést. Az osztályeloszlás az előfeldolgozási lépések után is közel változatlan maradt.

Az üzenethosszak karakteralapú eloszlása erősen jobbra ferde volt, ezért a kiugró értékek meghatározása a **Tukey interkvartilis módszer** (Tukey's Interquartile Range Method) alapján történt. A módszer az interkvartilis tartomány segítségével határozza meg az alsó és felső határokat a kiugró értékek azonosítására. A karakterhossz jó közelítést ad az üzenetek információtartalmára, ezért az extrém rövid és extrém hosszú üzenetek eltávolítása indokolt volt.

Az alsó határ esetében a módszer negatív értéket eredményezett, amely gyakorlati szempontból nem értelmezhető, ezért minimális küszöbértékként 15 karakter került meghatározásra. Ennél rövidebb üzenetek biztosan nem tartalmaznak elegendő szemantikai információt az osztályozáshoz. A felső határ újraszámítása a tanítóhalmazon történt, amely alapján a végső küszöbérték 2452 karakter lett. Ennek megfelelően eltávolításra kerültek a 15 karakternél rövidebb és a 2452 karakternél hosszabb üzenetek.

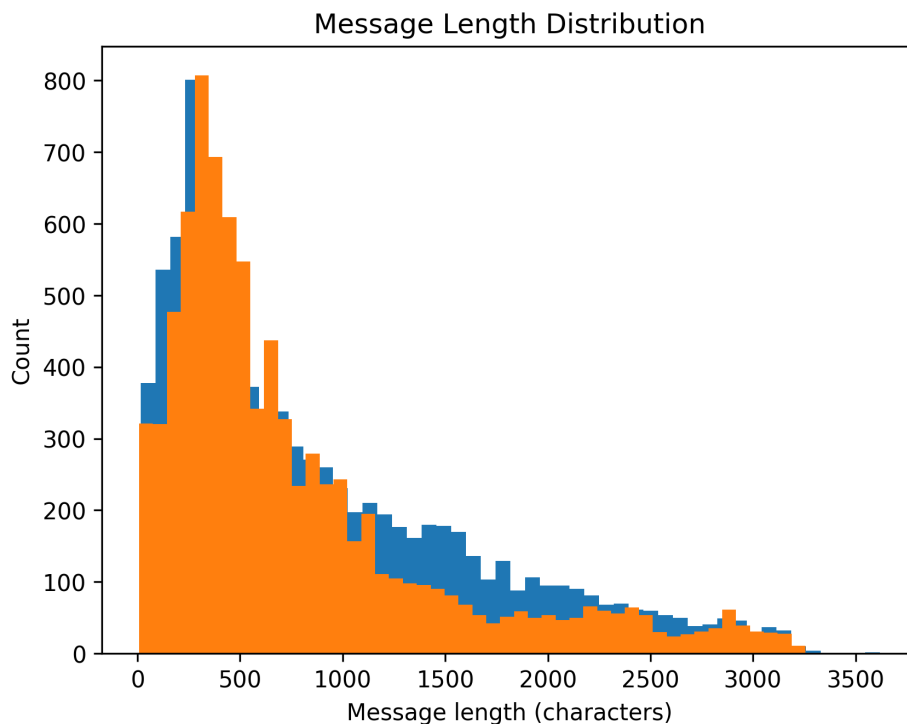
Az URL-ek, e-mail címek, telefonszámok és számok maszkolásra kerültek speciális tokenek segítségével (<URL>, <MAIL>, <PHONE>, <NUM>). A maszkolás célja a szókészlet méretének csökkentése úgy, hogy közben az adott mintázatok szemantikai szerepe megmaradjon.

Az ismétlődő karakterek egységesítése szintén a szókészlet méretének csökkentését szolgálta. Amennyiben egy karakter legalább háromszor ismétlődött egymás után, az

ismétlések száma kettőre került csökkentésre. A speciális karakterek végül nem kerültek eltávolításra, mivel jelenlétük fontos információt hordozhat a kéretlen üzenetek felismerése során.

A mintázatok helyettesítése reguláris kifejezések alkalmazásával történt, amelyek a konfigurációs fájlban kerültek rögzítésre. A helyettesítési folyamat nehézségét az jelentette, hogy az adathalmazban számos speciális karakter körül szóközök jelentek meg, amelyek akadályozták a reguláris kifejezések pontos működését. Ennek kezelésére további normalizációs lépések történtek, amelyek eltávolították a felesleges szóközöket, majd a mondatvégi írásjelek után újra beszúrták azokat.

Az előfeldolgozás eredményeinek ellenőrzése a `3-verification.ipynb` notebookban történt. Az eredeti 33 716 üzenetből összesen 4 311 került eltávolításra, így a végleges korpusz 29 405 üzenetet tartalmazott. A spam és ham címkék aránya mindhárom részhalmazban egy százalékon belüli eltérést mutatott, valamint az üzenethossz-eloszlás alakja sem változott jelentősen. A tanítóhalmaz előfeldolgozás utáni karakterhossz-eloszlását a 3.3. ábra szemlélteti.



3.3. ábra: Előfeldolgozás utáni üzenethossz-eloszlás karakterszám alapján.

Megfigyelhető, hogy az előfeldolgozás után néhány, az eredeti felső határnál hosszabb üzenet továbbra is jelen van. Ennek oka, hogy bizonyos mintázatok maszkolása megnövelte az üzenetek karakterszámát. Például nagyszámú numerikus érték esetén minden szám a `<NUM>` tokenre került cserélésre, amely hosszabb karakterláncot eredményezett.

Az utolsó lépés a szókészlet méretének becslése volt. A becslés az összefüggő alfanumerikus karaktersorozatok vizsgálata alapján történt. Az adathalmaz összesen 82 789 különböző egyedi alfanumerikus szekvenciát tartalmazott, azonban ezek közül 51 717 darab (62.47%) háromnál kevesebbszer fordult elő. Ez alapján a becsült szókészletméret körülbelül 31 072 token lett, amely megfelelő kiindulási alapot biztosított a tokenizer konfigurációjához.

3.3.5. Tokenizálás és validálása

A tisztítás után következett a tokenizáció, amelynek forráskódja a 4-tokenization.ipynb fájlban található. A notebook bemenetét a feldolgozott tanító-, validációs- és tesztadathalmazok képezték. Kimenetként előállításra került a SentencePiece tokenizer modellje, a generált tokenkészlet, valamint az egyes szövegek tokenazonosító-sorozatai pickle formátumban.

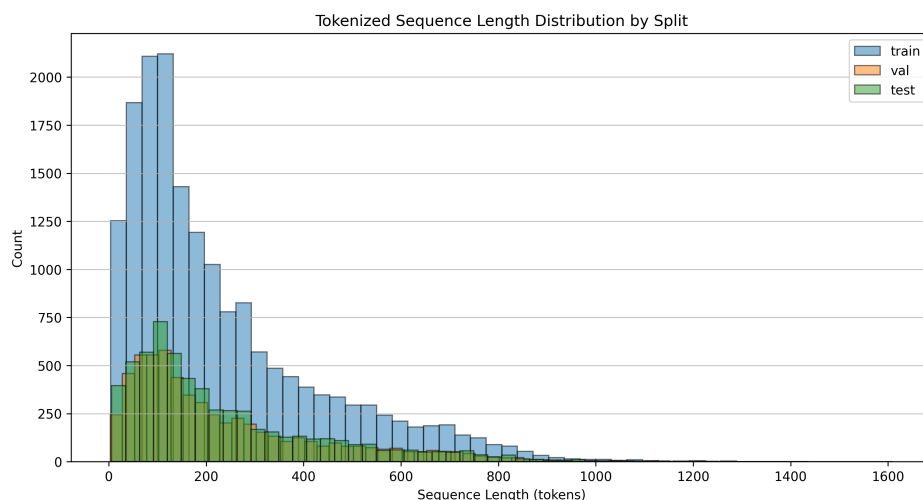
A tokenizáló tanítását a `train` metódus biztosította, az alábbi kódrészleten látszódik, hogy ebben a lépésben került meghatározásra a szókészlet mérete, melynek értéke 30.000 (különböző) token volt. A `user_defined_symbols` paraméterben kerültek megadásra a speciális szimbólumok, amelyeket a tokenizáló modellnek nem kellett tovább bontania. Az Unigram modell előnye az volt, hogy az ismeretlen szavakat jól kezelte, rugalmas részsavas szegmentálást biztosított, és az `character_coverage` 1.0 értéke biztosította, hogy az összes karakter feldolgozásra került.

Programkód 3.1. Példa: tokenizálás memóriában

```
spm.SentencePieceTrainer.train(
    input=str(PATHS.lstm_tokenizer_train_data),
    model_prefix=tokenizer_model_prefix,
    vocab_size=30000,
    user_defined_symbols=["<MAIL>", "<URL>", "<NUM>", "<PHONE>"],
    model_type='unigram',
    character_coverage=1.0
)
```

A tokenizált adathalmazok `pickle` formátumban kerültek mentésre, így könnyen visszatölthetők voltak memóriába. Az `ids.pkl` fájlok a numerikus azonosítókat, a `pieces.pkl` fájlok pedig a szöveges tokenrészleteket tárolták. Az objektumok lemeze mentésével nem kellett a tokenizálást minden futtatásnál újra elvégezni.

A tokenizálás validálásához a `./notebooks/5-validation.ipynb` fájlt vizsgáltuk meg, ahol a három adathalmaz hossz eloszlása szintén kirajzolásra került, itt azonban a tokenek száma alapján, és az A 3.4 ábrán látható.



3.4. ábra: A tokenizált adatok hosszának eloszlása token szám alapján.

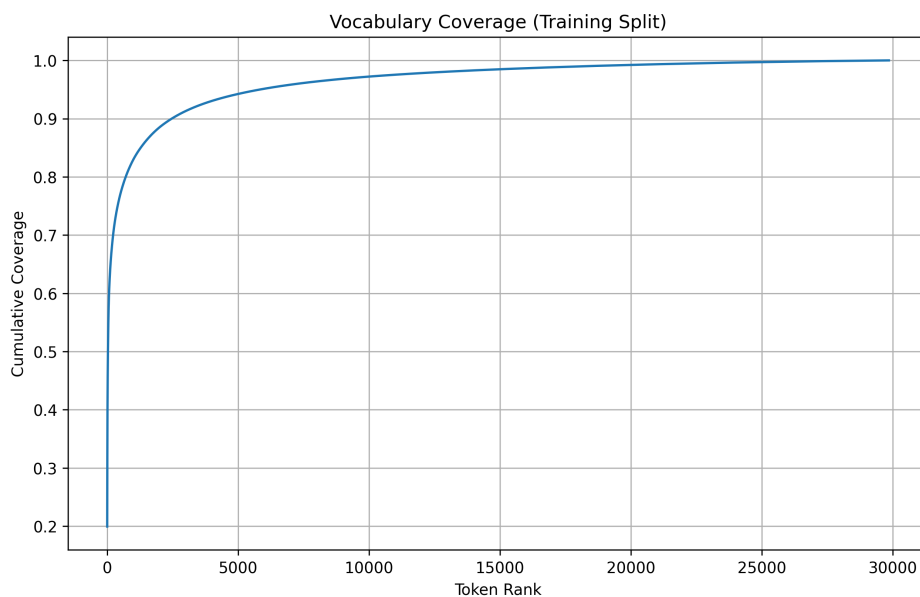
A tokenizált tanítóhalmazon szókészlet-lefedettségi elemzés is készült (3.5. ábra)

annak érdekében, hogy jobban megérthető legyen a tokeneloszlás és a tokenizáló működése. Az elemzés során először meghatározásra kerültek az egyes tokenek előfordulási gyakoriságai, majd ezek alapján kiszámításra került a kumulatív lefedettségük is. Az eredmények azt mutatták, hogy a leggyakrabban előforduló tokenek viszonylag kis része a teljes tokenállomány jelentős hányadát lefedte, ami jól illeszkedik a természetes nyelvekre jellemző Zipf-eloszláshoz.

Zipf-törvény egy empirikus statisztikai törvény, amely szerint számos természetes eloszlás, egy elem előfordulási gyakorisága fordítottan arányos annak gyakorisági rangsorban elfoglalt helyével [11].

A vizsgálat alapján az első harminc leggyakoribb token már a teljes tanítóhalmaz tokenjeinek körülbelül ötven százalékát lefedte. Ez arra utal, hogy a tokenizálás megfelelően működött, és a modell számára sikerült azonosítani a nyelvben leggyakrabban előforduló elemeket. A leggyakoribb tokenek listája külön is kiíratásra került annak ellenőrzésére, hogy azok nyelvileg és technikailag is értelmezhető mintázatokat képviselnek-e.

Nem meglepő módon a leggyakoribb token a szóköz karakter volt, amely aláhúzással került reprezentálásra. Az első tíz leggyakoribb token között számos speciális karakter is megjelent, például a vessző és a kettőspont, továbbá itt szerepeltek a szám- és URL-maszkok is. A lista további részében már gyakori rövid szavak és szórészletek jelentek meg, például a *your*, *of*, valamint az olyan gyakori tokenrészletek, mint a *_to* és a *_ing*. Ezek az eredmények összességében arra utalnak, hogy a tokenizáló képes volt a nyelv gyakori mintázatait és szerkezeti elemeit megfelelően reprezentálni.



3.5. ábra: A szókincs növekményes lefedettségi elemzése a tanító halmazon.

A gyakori, önálló jelentéstartalommal általában nem rendelkező szavak, az irodalomban úgynevezett **stop-szavak** (stop words), nem kerültek eltávolításra. Ennek oka, hogy ezek helyes és természetes használata fontos nyelvi mintázatokat hordozhat, amelyek hozzájárulhatnak az üzenetek jóindulatú vagy legitim jellegének felismeréséhez. Emiatt a leggyakrabban előforduló tokenek jelentős része ilyen stop-szó volt.

4. fejezet

A klasszifikációs modell és tanítási folyamat

4.1. Modelltervezés

A dolgozatban alkalmazott klasszifikációs modell egy kétirányú hosszú rövid távú memóriával rendelkező neurális architektúra, amely bináris szövegklasszifikációs feladatot valósít meg. A modell célja annak meghatározása, hogy egy elektronikus levél kéréslen vagy valódi üzenet kategóriába tartozik-e.

A modell bemenetét tokenazonosítók sorozata képezi, amelyeket a korábban ismertetett SentencePiece tokenizáló állít elő. Kimenete pedig egy 0 és 1 közé eső szám, az annak a valószínűsége, hogy a szekvencia kéréslen üzenet. Egy küszöbvel (értéke 0.5) bináris döntést hozunk. Ha a valószínűség nagyobb, mint 0.5, akkor kéréslen az üzenet.

4.1.1. Modell architektúra

A kéréslen üzenetek bináris osztályozására egy kétirányú hosszú rövid távú memória neurális hálón alapuló architektúra használata mellett esett a döntés. A modell célja a szöveges üzenetek szekvenciális reprezentációjának hatékony feldolgozása a szövegkörnyezet mindkét irányból való figyelembevételével.

A bemenetek változó hosszúságúak, ezért feldolgozás során **kitöltés** (padding) alkalmazása szükséges, és ezek a kitöltő tokenek ne járulhassanak hozzá a tanulható tokenekhez. A beágyazási réteg feladatai közé tartozik ezen tokenek kezelése, mely az első réteg az architektúrában. Ebben a rétegben történik a token azonosítók egész számokból való kontextusban gazdag vektorra alakítása. Ezt követően jönnek a hosszú rövid távú memória rétegek, ezek kimenete az utolsó réteg mindkét irányú rejtett állapotainak összefűzése.

Ezt a kimenetet két **teljesen összekapcsolt** (Dense) lineáris réteg követi, amelyek feladata az osztályozás; közöttük egy **egyenirányított lineáris egység** (Rectified Linear Unit, ReLU) aktivációs függvény és egy **lemorzsolás** (**dropout**) regularizáció kerül alkalmazásra a nemlinearitás biztosítása és a túlilleszkedés csökkentése érdekében, a modell általánosító képességének javítására. A második teljesen összekapcsolt réteg kimenete egyetlen neuront tartalmaz, amely logit értéket állít elő.

4.1.2. Hiperparaméter-tér és optimalizálási stratégia

A megfelelő paraméterek kiválasztása az F1 érték maximalizálása szerint történik random hiperparaméter-keresés során az előre definiált értéktartományokból [1] pszeudo-véletlenszerűen kerül kiválasztásra az érték minden paraméter esetében a reprodukálhatóság érdekében. Az optimalizáció alapértelmezett célfüggvénye az F1 mérőszám, melynek jelentősége kiegyensúlyozatlan osztályozás esetén nagyobb. Helyes választás lehet a precizitás vagy a visszahívás is, a megoldandó feladattól függően. Magas precizitás esetén, ha a "pozitív" osztályunk a kéretlen üzenet, kevés normális üzenet kerül a spam mappába, ez a valósághűbb működés. Míg a visszahívás arra ad választ, hogy a valódi kéretlen üzenetek mekkora részét találja meg a modell. Az F1-érték melletti döntést az általános kéretlen üzenetek detektálása támasztja alá.

Ezekkel a paraméterekkel egy új modell kerül betanításra és kiértékelésre. A tanítás folyamata a következő alfejezetben kerül bemutatásra.

A keresési tér kialakításának célja olyan hiperparaméter-intervallumok meghatározása volt, amelyek egyensúlyt biztosítanak a modell reprezentációs kapacitása, a tanítás stabilitása és a számítási költség között. A vizsgált tartományok nem univerzális optimális értékek, hanem a szakirodalomban és korábbi BiLSTM-alapú szövegosztályozási megközelítésekben alkalmazott konfigurációk, valamint a feladat sajátosságai alapján meghatározott kísérleti keresési tér részei [2, 13].

- **Maximális szekvenciahossz** A bemeneti szekvencia maximális hossza meghatározza, hogy a modell mennyi kontextuális információt képes feldolgozni egy mintából. A túl rövid szekvenciák információvesztéshez vezethetnek, míg a túl hosszú szekvenciák jelentősen növelik a memória- és számítási igényt. A korábbi statisztikai elemzés alapján a bemeneti szövegek túlnyomó része 500–800 token között helyezkedik el, ezért a vizsgált intervallum 500–1000 token volt.
- **Batch méret** A batch size az egy iteráció során feldolgozott minták számát határozza meg. Kis batch méret zajosabb gradiensbecslést eredményez, míg túl nagy batch esetén a generalizáció romolhat a csökkent gradiensvariancia miatt. A szakirodalomban gyakran alkalmazott tartományokkal összhangban a vizsgált értékek 32, 64 és 128 voltak [2].
- **Szóbeágyazási dimenzió** Az embedding dimenzió a tokenek sűrű vektorreprezentációjának méretét határozza meg. Alacsony dimenzió esetén a reprezentáció kapacitása korlátozott, míg túl magas dimenzió esetén a modell paraméterszáma és a túltanulás kockázata növekszik. A vizsgált értékek 64, 128 és 256 voltak, összhangban korábbi BiLSTM-alapú szövegklasszifikációs megközelítésekkel [2, 13].
- **Rejtett réteg mérete** A LSTM rejtett állapotának dimenziója közvetlenül befolyásolja a modell reprezentációs kapacitását. A kisebb méret gyorsabb tanítást eredményez, míg a nagyobb dimenzió jobb reprezentációt biztosíthat, de növeli a túltanulás és a számítási költség kockázatát. A vizsgált értékek 64, 128, 256 és 384 voltak, a korábbi szövegklasszifikációs BiLSTM architektúrák gyakorlati beállításai alapján [13].
- **Rejtett rétegek száma** Az egymásra épített LSTM rétegek száma a modell mélységét határozza meg. A mélyebb architektúrák elméletileg nagyobb reprezentációs kapacitást biztosítanak, ugyanakkor érzékenyebbek a túlilleszkedésre és

nehezebben optimalizálhatók. A vizsgált konfigurációk 1–3 réteget tartalmaztak, összhangban korábbi BiLSTM alapú szövegosztályozási modellekkel [2, 13].

- **Dropout arány** A dropout regularizációs technika, amely a neuronok véletlenszerű elhagyásával csökkenti a túlilleszkedést és javítja a generalizációt. Alacsony dropout érték esetén a regularizáció hatása gyenge, míg túl magas érték esetén a modell alultanulhat. A vizsgált intervallum 0.15–0.55 volt, amely a korábbi BiLSTM szövegklasszifikációs modellekben alkalmazott tartományokkal összhangban van [2].
- **Teljesen összekapcsolt réteg mérete** A kimeneti osztályozó sűrű rétegének mérete a végső reprezentációs transzformáció kapacitását határozza meg. Kis méret esetén a döntési tér korlátozott, míg nagy méret esetén a túlilleszkedés kockázata növekszik. A vizsgált értékek 32, 64 és 128 voltak, a korábbi BiLSTM alapú osztályozási architektúrák gyakorlati beállításai alapján [2].
- **Tanulási ráta** A tanulási ráta az optimalizációs lépések nagyságát határozza meg. Túl alacsony érték esetén a konvergencia lassú, míg túl magas érték esetén a tanítás instabillá válhat. A vizsgált tartomány 5×10^{-5} és 3×10^{-3} között került meghatározásra, logaritmikus skálán mintavételezve. [3].
- **Maximális gradiensnorma** A gradiensklippelés célja a robbanó gradiensnek hatásának mérséklése, különösen rekurzív neurális hálózatok esetén. A túl alacsony küszöb korlátozhatja a tanulást, míg túl magas érték esetén a klippelés hatása elhanyagolható. A vizsgált intervallum 0.2–3.0 volt, amely a stabil tanítási gyakorlatokban alkalmazott tartományokkal összhangban került meghatározásra.
- **Epochok száma** Az epochok száma a teljes tanítóhalmazon történő áthaladások számát határozza meg. Alacsony epoch szám esetén a modell alultanulhat, míg túl magas érték esetén a túlilleszkedés kockázata nő. A vizsgált tartomány 2–5 epoch volt, korai megállítás (early stopping) és validációs teljesítmény monitorozása mellett [13].

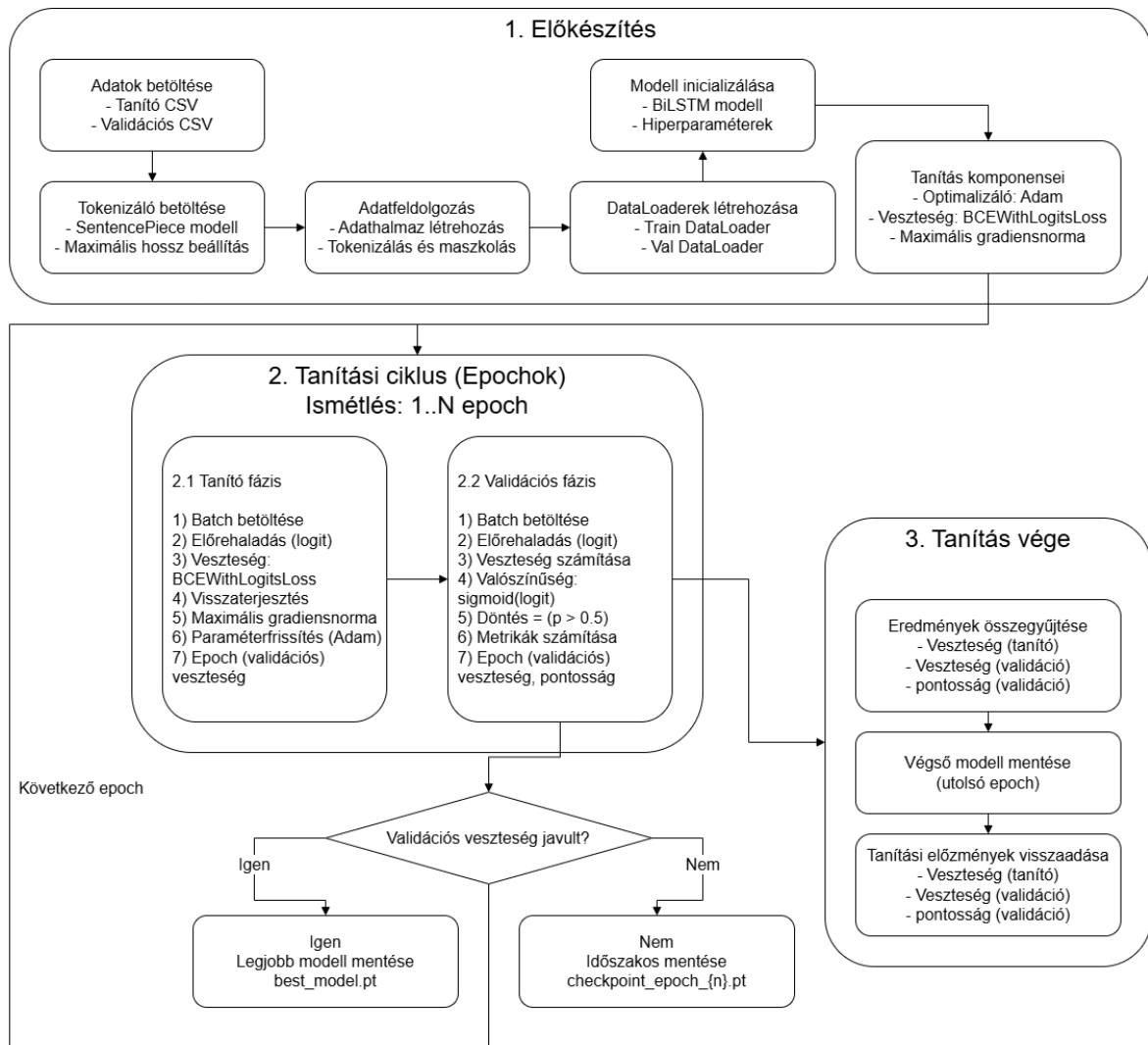
A diszkrét hiperparaméterek esetében előre definiált értékkészletből történt a mintavételezés, míg a folytonos paramétereknél intervallumalapú keresést alkalmaztunk. A tanulási ráta logaritmikus skálán került mintavételezésre, mivel annak optimális értéke jellemzően több nagyságrendet is lefedhet.

Tanítási folyamat, hangolás és tanítóhalmaz növelés

A tanítási folyamat tulajdonképpen koncepcionálisan teljesen megegyezik a hangolás és a növekményes tanítóhalmaz vizsgálata között. Az eltérés logikusan abból áll, hogy a hiperparaméterek keresése során a tanító-, validációs- és tesztadathalmazok változatlanok, míg a hiperparaméterek és az előző fejezetben megállapított rétegek mérete és számossága véletlenszerűen változnak a megadott értéktartományjaiban. Míg a másik esetben a tanítóhalmaz mérete növekszik kumulatíván, más néven halmozódóan, miközben a hiperparaméterek és a többi adathalmaz változatlan marad. A 4.1 ábra

⁰A rétegek számosságát és méretét, valamint az epochok számát a [13] tanulmány inspirálta, amely rövid szövegek bináris szentimentklasszifikációjára fókuszált. A jelen feladat hosszabb szövegek feldolgozását igényli, ezért a keresési tér felső határai tágabbak voltak. Ugyanakkor a számítási erőforrások korlátai és a dataset mérete indokolták az epochok alacsonyabb tartományban történő vizsgálatát.

szemlélteti a tanítási folyamatot, mely az előkészítés, tanítás és kiértékelés részekből áll.



4.1. ábra: A tanítás folyamata részletesen.

4.2. Implementáció

A modell, a tanítási folyamat és minden egyéb segédprogram implementálása Python nyelven történt, és az egész projekt egységesen a Python 3.10.0 verziót használja. A kódbázis szerkezetét és egyes rövidebb kódrészleteket a BitNinja Technologies Zrt. zárt hozzáférésű projektje inspirálta, melyből részlet nem kerülhet bemutatásra. A kód futtatásához a következő, pip-pel telepíthető külső csomagok kerültek felhasználásra (zároljellenben a dokumentációban feltüntetett verziószámokkal):

- pandas (2.3.3);
- matplotlib (3.10.8);
- scikit-learn (1.7.2);

- sentencepiece (0.2.1);
- numpy (2.2.6).
- torch (2.11.0+cu130);

Továbbá a kód objektumorientált programozási paradigma alapján készült, melynek következtében a kód modularitása kedvezően befolyásolja a kód olvasását, karbantartását és esetleges továbbfejlesztését.

A kezelő felület nagyon egyszerű karakteres felület. Két fő argumentuma a `tune` és a `learning-curve`. A `tune` argumentum esetén a `-num-trials` nevű opcionális argumentum a 20 értéket kapja alapértelmezetten mely a hiperparaméter-keresési próbálkozások száma, míg a `learning-curve` argumentum esetén a `-num-portions` nevű opcionális argumentum a 10 értéket kapja alapértelmezetten, amely a tanítóhalmaz méretének növelése során végzett vizsgálatok száma, és e szám alapján kerül a tanítóhalmaz felosztásra.

4.2.1. Állomány és konfigurációk kezelése

Az állománykezelési műveletek megvalósításáért az `ArtifactManager` osztály felel. Az osztály egységes interfészt biztosít különböző típusú adatok mentésére és betöltésére, beleértve:

- PyTorch modellek és checkpointok mentését,
- JSON konfigurációs állományok kezelését,
- CSV adathalmazok betöltését és mentését,
- grafikonok és ábrák exportálását,
- szöveges állományok kezelését.

Az osztály automatikusan kezeli a könyvtárstruktúra létrehozását, így a mentési műveletek során nem szükséges manuálisan létrehozni a hiányzó mappákat. Az implementáció támogatja a relatív és abszolút útvonalak egységes kezelését is.

A konfigurációk JSON formátumban kerültek tárolásra, melyek közül az egyik feladata az útvonalak és reguláris kifejezések tárolása, míg a másik kettő a hiperparaméterteret és a random hiperparaméter-keresés eredményeit tárolja. Az útvonalak kezelése nem teljes lefedettségű, találni a kódbázisban beégetett vagy kibővített útvonalakat. Az adatok betöltését és mentését külön Python osztályok kezelik.

4.2.2. Modell, hiperparaméterek és tanulási görbe

A modell a PyTorch keretrendszer használatával készült, és a `BiLSTMSpamClassifier` osztály tartalmazza architektúráját, amely az **előrecsatolás** (forward pass) "forward" nevű metódusából olvasható ki. Ez az egyetlen használt metódus a konstruktoron kívül. Szerkezete a Modell architektúra részben rögzítésre került.

A `RandomSearchTuner` osztály feladatai közé tartozik a hiperparaméter térből betöltött különböző érték tartományok kezelése, melyek közé került diszkrét lehetőség,

egészszámú vagy lebegőpontos intervallum illetve logaritmikus skálán való mintavételezés. Minden próba esetén egy új pszeudovéletlen hiperparaméter-konfiguráció generálódott, és ezekkel egy új modell tanítása kezdődött.

A tanítás folyamata mind a hiperparaméter mind a tanulási görbe esetében megegyezik, és futtatásukat az LSTMTrainingPipeline osztály indítja. Itt kerül példányosításra az összes különböző osztály, és `run()` metódusát hívja a mindkét feladat futtatása.

A hiperparaméter-keresés eredményei külön jegyzékekbe kerültek mentésre, és a következő adatokat tartalmazzák:

- a betanított modell állapot mentését,
- a tanítási előzményeket,
- a kiértékelési metrikákat,
- a konfúziós mátrixot,
- valamint a tanulási görbét.

A tanulási görbe osztálya a LearningCurveRunner osztály, feladata a tanítóhalmaz méretének növelése mellett az osztályozási teljesítmény vizsgálata és ezen vizsgálatok mentése vizuálisan és JSON formátumban, a már említett állomány kezelő osztály segítségével.

A kumulatívfelosztást végző külső segédfüggvény érdekessége, hogy a scikit-learn könyvtár StratifiedKFold osztályának `split` metódusát alkalmazza, mely pszeudorandom módon megkeveri az adatokat és biztosítja a közel azonos osztályeloszlást a részhalmazok között.

5. fejezet

Kísérletek és eredmények

5.1. Hiperparaméter-optimalizálás és tanulási görbe

Ebben a fejezetben a hiperparaméter-optimalizálás folyamata, valamint a kumulatív tanítóhalmazos tanulási görbe elemzése kerül bemutatásra. A vizsgálat elsődleges célja annak meghatározása volt, hogy milyen hiperparaméter-konfiguráció mellett képes a modell stabil és reprodukálható teljesítményt nyújtani, illetve hogyan változik az általánosító képessége a tanítóhalmaz méretének fokozatos növelésével.

A hiperparaméter-optimalizálás során nem kizárólag a lehető legmagasabb teljesítmény elérése volt a cél. Bár a paraméterek végső kiválasztása a maximális F1-érték alapján történt, a stabil modellviselkedést a keresési tér szűkítésével lehetett elérni. Az F1-érték a precizitás és a visszahívás harmonikus átlaga, érzékeny az osztályozási torzításokra, és kiegyensúlyozott képet ad a modell teljesítményéről.

A teljes optimalizálási folyamat három egymásra épülő fázisban valósult meg. Az első fázis célja a széles hiperparaméter-tér feltérképezése volt annak érdekében, hogy meg lehessen állapítani a paraméterter megfelelő beállítását. A második fázisban a keresési tér szűkítésére került sor, amelynek célja a stabilabb és hatékonyabban futtatható konfigurációk előtérbe helyezése volt. A harmadik fázisban már egy tudatosan konzervatívabb keresési stratégia került alkalmazásra, amely a tanulási görbe vizsgálatához szükséges stabil modellviselkedést támogatta.

5.1.1. Kísérleti környezet

A futtatások egyetlen lokális gépen történtek, a mérések során használt hardver és környezet a következő volt:

- **GPU:** NVIDIA GeForce RTX 5050 Laptop GPU,
- **GPU memória:** 8151 MiB VRAM,
- **Rendszermemória:** 33617924096 bájt (kb. 31,3 GiB),
- **Python:** 3.10.0,
- **PyTorch:** 2.11.0+cu130,
- **CUDA runtime:** 13.0.

Erőforrás-igény és futásidő-metrikák nem kerültek rögzítésre.

5.1.2. A három fázis keresési tere

A három kísérleti fázis közötti legfontosabb különbséget a hiperparaméter-tér fokozatos szűkítése jelentette. Az egyes iterációk során a korábbi futások tapasztalatai alapján kerültek módosításra a keresési tartományok annak érdekében, hogy csökkenjen az instabil vagy aránytalanul erőforrás-igényes konfigurációk száma. A 5.1 táblázat összefoglalja a három fázisban alkalmazott keresési tereket.

5.1. táblázat: A hiperparaméter-tér módosítása a három fázisban.

Hiperparaméter	1. fázis	2. fázis	3. fázis
Max. szekvenciahossz	[512, 800, 1024]	[500, 600, 700, 800]	[500, 600, 700]
Batch méret	[32, 64, 128]	[32, 64, 128]	[32, 64, 128]
Beágyazási dimenzió	[64, 128, 256]	[64, 128, 256]	[64, 128, 256]
Rejtett rétegek mérete	[64, 128, 256, 384]	[64, 128, 256]	[64, 128]
Rejtett rétegek száma	[1, 2, 3]	[2, 3]	[2]
Dropout arány	[0.15, 0.50]	[0.15, 0.50]	[0.15, 0.50]
Sűrű réteg mérete	[32, 64, 128]	[32, 64, 128]	[32, 64, 128]
Learning rate	[0.00005, 0.003]	[0.00005, 0.003]	[0.00005, 0.002]
Maximális gradiens norma	[0.2, 3.0]	[0.5, 3.0]	[0.5, 3.0]
Epochok száma	[2, 3, 4, 5]	[2, 3, 4]	[2, 3]

A második fázisban a cél elsősorban a szélsőségesen lassú konfigurációk kizárása volt, míg a harmadik fázisban már egy tudatosan konzervatívabb, kisebb varianciát és stabil működést eredményező keresési tér került kialakításra. Ennek eredményeként jelentősen csökkent az olyan konfigurációk aránya, amelyek instabil tanulási viselkedést vagy aránytalanul magas futási költséget okoztak.

5.1.3. Első fázis: kezdetleges keresési tér

Az első fázis célja egy széles hiperparaméter-tér feltérképezése volt, amelynek során azt vizsgáltuk, hogy a választott BiLSTM architektúra milyen teljesítményszinteket képes elérni különböző beállítások mellett. Ebben a szakaszban a hangsúly nem a végző optimalizáláson vagy a futási hatékonyságon volt, hanem a keresési tér általános viselkedésének és a különböző hiperparaméter-kombinációk hatásának feltárásán.

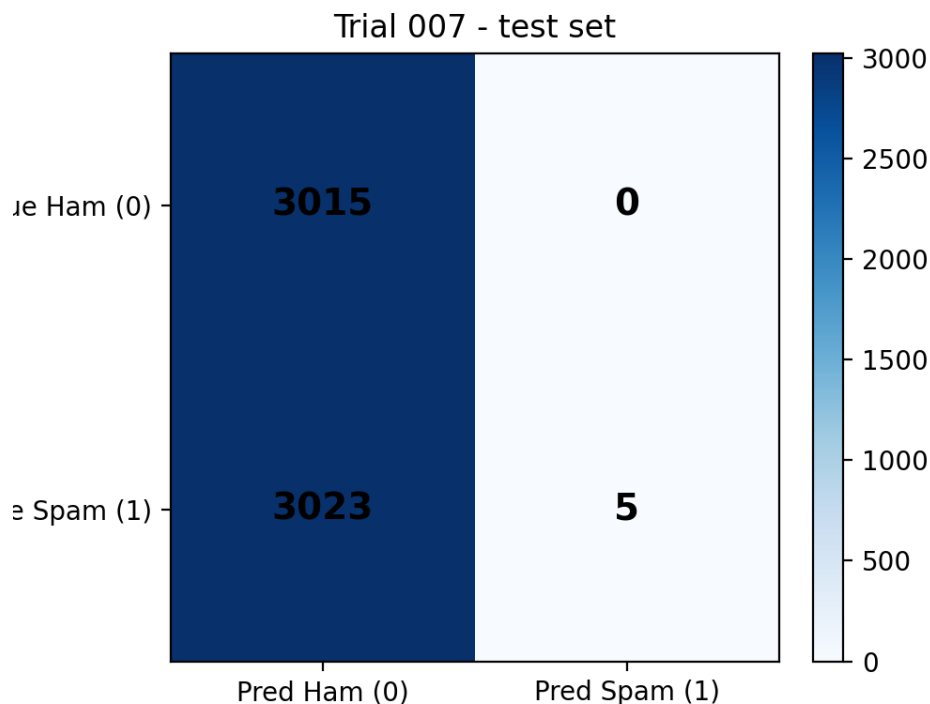
A futtatási sorozat gyakorlati okokból nem volt teljes mértékben végigvihető: a tizedik kísérlet futási ideje meghaladta a négy órát, ezért a folyamat manuálisan megszakításra került. Ez az eset rávilágít arra, hogy a hiperparaméter-optimalizáció során nem elegendő kizárólag a teljesítménymetriák javítására törekedni, hanem a számítási erőforrások és a futási idő is meghatározó korlátot jelentenek.

A megszakítás ellenére összesen kilenc teljesen kiértékelt kísérlet állt rendelkezésre. Az eredmények alapján:

- **legjobb F1-érték:** 0.9807 (a hatodik kísérletben),
- **legrosszabb F1-érték:** 0.0033 (a hetedik kísérletben),
- **átlagos F1-érték:** 0.8514.

Különösen informatívnak bizonyult a 7. kísérlet eredménye. Ebben az esetben a precizitás értéke magas maradt, miközben a visszahívás közel nullára csökkent, ezért az F1-érték gyakorlatilag összeomlott. Ez a viselkedés arra utal, hogy a modell erősen egyoldalú döntési stratégiát alakított ki, így csak nagyon kevés mintát sorolt a pozitív osztályba.

Ez a jelenség gyakran a mély hálók tanításának instabil dinamikájára vezethető vissza. Tipikusan túl agresszív tanulási ráta, nem megfelelő paraméter-inicializáció vagy instabil gradiensfrissítések okozhatják, amelyek megakadályozzák a modell konvergenciáját egy jól általánosító megoldás felé. Ennek következményeként a modell egy lokális "optimumban" reked, ahol ugyan bizonyos mintákon magas pontosságot ér el, de a teljes eloszlást nem tanulja meg megfelelően [3].

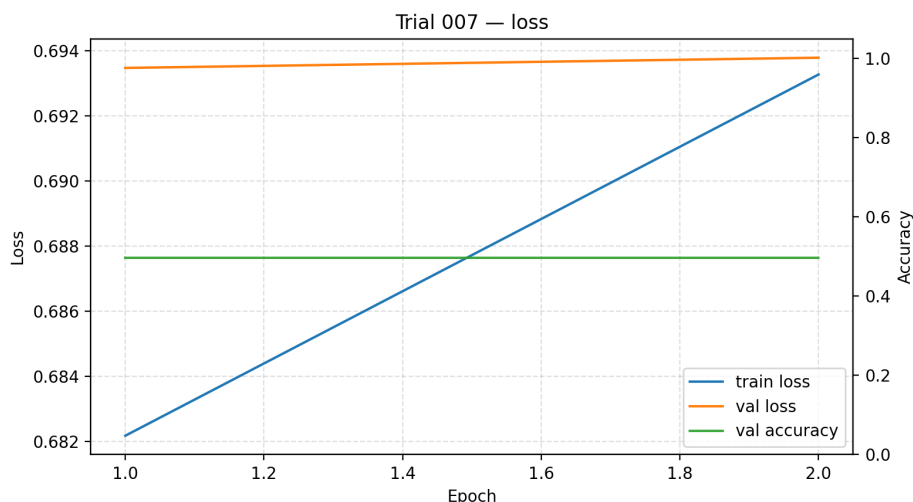


5.1. ábra: A 7. kísérlet konfúziós mátrixa. A modell egyoldalú döntése alapján a teszhalmazon öt darab kivétellel az összes üzenet helytelenül valódi üzenetként került felcímkézésre.

A konfúziós mátrix (5.1) és a tanulási görbe (5.2) egyaránt azt mutatja, hogy a pontosság (accuracy) önmagában nem elegendő a modell teljesítményének értékelésére. Bizonyos esetekben a modell úgy is képes körülbelül 0.5-ös pontosságot elérni, hogy szinte kizárólag egyetlen osztályt prediktál, ami erősen torzított osztályozási viselkedésre utal [3, 11. fejezet.].

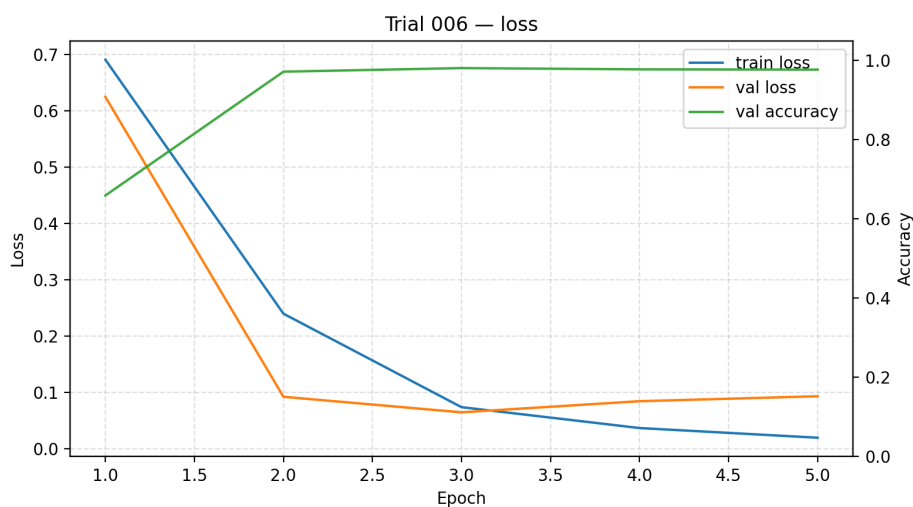
A **tanulási veszteség** (training loss) a tanító adathalmazon, részhalmazokra bontva számított veszteségfüggvény-érték, amely a modell predikciói és a valódi címkék közötti eltérést méri, és a tanulási folyamat során a súlyok frissítésének alapjául szolgál. Ezzel szemben a **validációs veszteség** (validation loss) a validációs adathalmazon számított veszteség, amely a modell általánosító képességének értékelésére szolgál, és nem vesz részt a paraméterek optimalizálásában.

A legrosszabb tanítási futás (5.2) és a legjobb futás (5.3) összehasonlítása alapján megfigyelhető, hogy a megfelelően konvergáló modell esetében mind a tanulási, mind



5.2. ábra: A `trial_007` tanulási görbéje. Leolvasható a pontosság értéke, amely önmagában nem jelenti a tanítás sikerességét.

a validációs veszteség egyértelműen csökkenő irányt mutat, ami stabil tanulásra és jó általánosító képességre utal. Ezzel szemben a gyengébb teljesítményű futás során a veszteségértékek nem mutattak állandó javulást, hanem mind a tanulási, mind a validációs veszteség kis mértékű növekedése volt megfigyelhető, ami instabil optimalizációra és nem megfelelő konvergenciára utal.



5.3. ábra: A 6. kísérlet tanulási görbéje. Leolvasható a pontosság értéke, amely önmagában nem jelenti a tanítás sikerességét.

5.1.4. Második fázis: szűkített tér

A második fázisban a keresési tér szűkítésével összesen 20 kísérlet teljes lefuttatására került sor, amely lehetővé tette a növekményesen bővített tanítási halmazhoz tartozó tanulási görbék részletes vizsgálatát is, illetve a modellek F1-score alapján kirajzolt tanulási görbéi is elkészültek. A cél ebben a szakaszban már nem a teljes hiperparaméter-

tér feltérképezése volt, hanem egy szűkebb, számításilag hatékonyabb keresési tér vizsgálata, amely várhatóan jobb általánosítási teljesítményt eredményez.

A keresési tér szűkítése során a hosszú-rövid távú memória (LSTM) rétegek számát és rejtett dimenzióját, a szekvenciahosszt, valamint az epochok számát csökkentettük. Ennek célja egyrészt a tanítási idő mérséklése, másrészt a túlilleszkedés (overfitting) kockázatának csökkentése volt. Ezzel párhuzamosan a gradiensvágás küszöbérték keresési tartománya is módosításra került, mivel a túl alacsony határérték a gradiens túlzott levágásához vezethet, ami jelentősen csökkenti a paraméterfrissítések mértékét és rontja a tanulás hatékonyságát.

Az öt legjobban teljesítő konfiguráció vizsgálata alapján megfigyelhető, hogy a szekvenciahossz eloszlása a kisebb értékek felé tolódik. A minták közül csupán egy esetben jelent meg a 800 tokenes maximális szekvenciahossz, míg a többi konfiguráció 500 és 700 tokenes beállításokat alkalmazott. Ez arra utalhat, hogy az osztályozáshoz szükséges információk jellemzően a szövegek korábbi szakaszaiban is rendelkezésre állnak, így a hosszabb szekvenciák elsősorban a számítási költséget növelik, rosszabb esetben zajt visznek a tanításba. Ezek alapján a maximális szekvenciahossz további csökkentése indokolt.

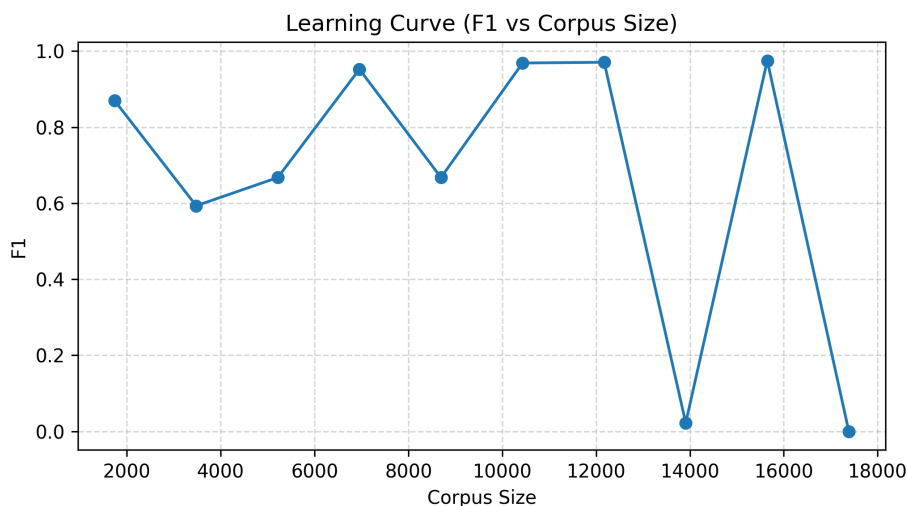
Hasonló figyelhető meg az LSTM rejtett rétegméret esetében is, ahol az esetek többségében a 128-as méret dominál, míg a nagyobb dimenziók ritkábban szerepelnek a legjobb konfigurációk között. Ez arra utalhat, hogy a modell kapacitása már ezen a szinten is elegendő a feladat reprezentálásához, és a további növelés nem feltétlenül eredményez teljesítményjavulást.

Az öt legjobban teljesítő konfiguráció összefoglalása az alábbi táblázatban látható.

5.2. táblázat: A top 5 konfiguráció hiperparaméterei (F1 szerint rendezve).

Hiperparaméter	trial_003	trial_001	trial_010	trial_007	trial_005
F1	0.9814	0.9729	0.9681	0.9657	0.9630
Max. szekvenciahossz	800	500	700	500	700
Batch méret	32	32	32	64	32
Embedding dimenzió	128	256	256	128	256
Hidden size	256	128	128	128	128
LSTM rétegek száma	3	2	3	3	2
Dropout	0.4333	0.2281	0.4596	0.2069	0.4906
Dense hidden	32	128	128	64	64
Learning rate	0.001117	0.000076	0.000327	0.000118	0.000069
Max grad norm	0.899	2.352	1.162	1.167	1.233
Epochok száma	3	4	2	4	4

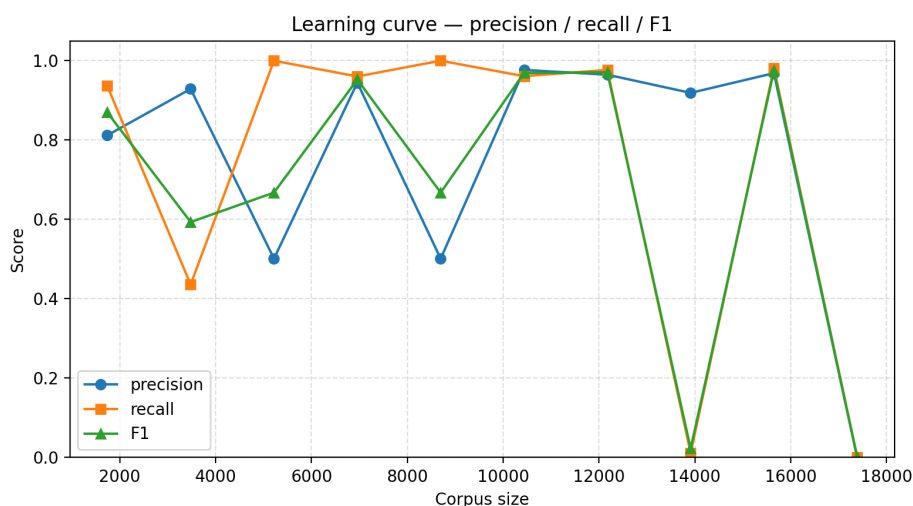
A top-5 konfigurációk elemzése után a következő lépésben a modellek viselkedését vizsgáljuk a növekményes tanítóhalmaz alkalmazása esetén. A 5.4 ábra azonos architektúrájú és hiperparaméterű modellek F1-értékét mutatja a tanítóadatok fokozatos növelése mellett.



5.4. ábra: Instabil modell növekményes tanulási görbéje az F1-érték szerint, a második fázisban.

A görbe alapján megfigyelhető, hogy a modell teljesítménye a tanítás kezdeti szakaszában jelentős ingadozást mutat, 0,6 és 0,9 közötti F1-értékekkel. A tanítóadatok mennyiségének növekedésével a teljesítmény részben stabilizálódik, és a görbe kisebb varianciát mutat.

A későbbi szakaszban azonban két alkalommal is megfigyelhető anomália, amikor az F1-érték gyakorlatilag nullára csökken. Ez a jelenség tipikusan akkor fordul elő, amikor a modell teljesen hibás predikciókat ad az adott szakaszban. Matematikailag ez azzal magyarázható, hogy az F1-score a precizitás (precision) és a visszahívás (recall) harmonikus átlaga, így amennyiben a precizitás vagy a visszahívás nullához közelít, az F1-érték is nullához tart. A 5.5 ábrán látható, hogy ebben az esetben a visszahívás értéke tartott a nullához, azaz a modell a teszhalmaz majdnem minden üzenetét a negatív osztályba sorolta. A negatív osztály ebben az esetben a valós üzeneteket jelenti.



5.5. ábra: Instabil modell növekményes tanulási görbéje az F1-érték szerint, a második fázisban.

Szintén beszédes mutató az egyes modellek tanítható paramétereinek száma. A

legrosszabb és legjobb konfiguráció közötti különbséget véve példaként arra következtethetünk, hogy a rejtett rétegek számát érdemes tovább csökkenteni, hiszen a hetedik kísérlet modelljének 4 189 313 tanítható paramétere több mint másfélszerese a hatodik kísérlet modelljének. Ez arra utal, hogy a modellkapacitás további növelése nem eredményezett arányos teljesítményjavulást, miközben jelentősen növelte a tanítható paraméterek számát és a számítási költséget.

Ez a fázis tehát azt igazolta, hogy a keresési tér átgondolt szűkítésével javítható a gyakorlati futtathatóság. Megfigyelhető továbbá, hogy a hiperparaméter-keresés során még a legrosszabb konfigurációk is elfogadható F1-értéket eredményeztek, ami arra utal, hogy a futások közötti variancia érdemben csökkent. Ugyanakkor bizonyos esetekben továbbra is instabil viselkedés volt tapasztalható.

5.1.5. Harmadik fázis: Általánosító képesség

A harmadik fázis célja a növekményes tanítóhalmazos tanulási görbe stabilizálása. Ehhez a keresési tér tovább szűkült, majd a kiválasztott konfigurációval a tanulási görbe (learning curve) futtatása következett.

A módosítások a második fázis vizsgálatára alapján kerültek megállapításra.

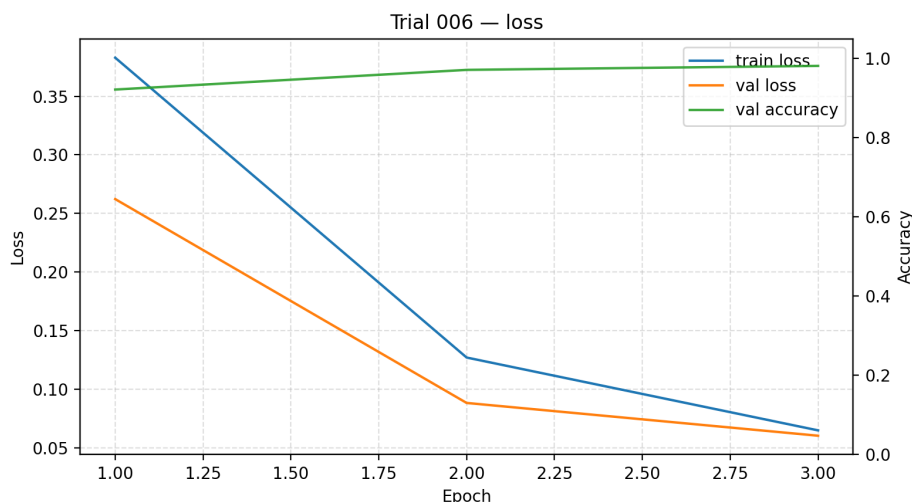
Az öt legjobb konfiguráció vizsgálatának tanulsága szerint a szekvencia túlzott hossza teljesítményromlást okozhat, egyúttal mindenképpen növeli a számítási költséget. Ezért a 500-, 600-, 700- és 800 tokenes maximális hosszak diszkrét tartományából eltávolításra került a 800-as érték.

Látható volt, hogy a magas paraméterszám mellett a modellek általánosító képessége csökkent. Így a rejtett rétegek száma fixen kettőre lett csökkentve, míg a méretük a 64 illetve 128 dimenzióra került korlátozásra.

Az epochok száma is csökkentésre került, így csökkentve a tanítási időt és a túlilleszkedés kockázatát. A 4 epochos beállítás a legjobb teljesítmények között is megjelent nagyszámban, azonban az eredmények alapján remélhető a kevesebb epoch melletti jó általánosító képesség a többi paraméter megfelelő értéke mellett. A további hiperparaméterek értékei:

Ezek a tartományok mellett a hatodik vizsgálat érte el a legjobb F1-értéket, mely 0.9817 volt. A modell 2 090 049 tanítható paraméterrel rendelkezett, ami a nagyobb konfigurációkhoz képest viszonylag alacsony paraméterszámnak tekinthető. A rétegek méretei a legalacsonyabb értékek a keresési térből (beágyazási - 64, rejtett 2db - 64, sűrű - 32) ennél a modellenél. A további hiperparaméterek értékei: batch méret - 32, dropout arány - 0.4948275322312652, tanulási ráta - 0.0012221621228809315, maximális gradiens norma - 1.4503155625979154, epochok száma - 3.

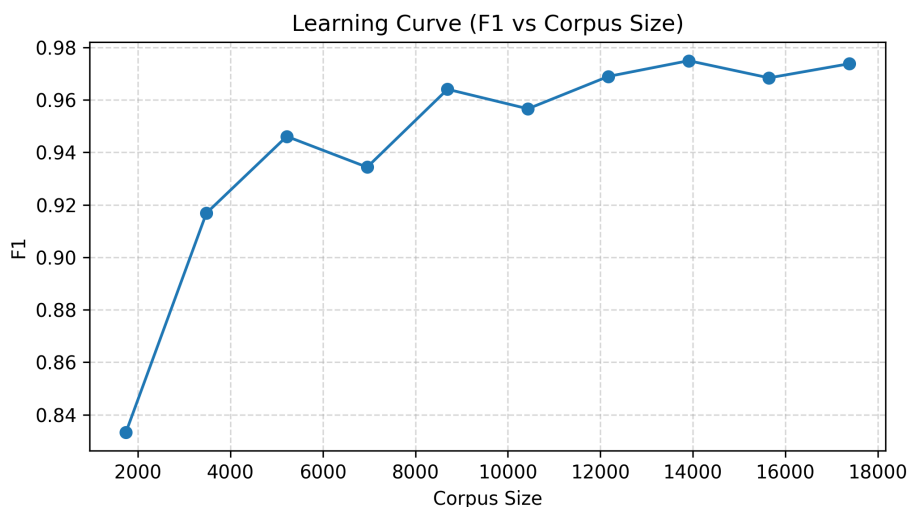
A viszonylag magas dropout valószínűleg hozzájárult a modell általánosító képességének javításához azáltal, hogy csökkentette a túlilleszkedés kockázatát. A tanulási görbék alapján megfigyelhető, hogy mind a tanulási, mind a validációs veszteség fokozatosan csökkent, miközben a validációs pontosság stabilan növekedett. Ez kiegyensúlyozott és stabil tanulási folyamatra utal.



5.6. ábra: A hatodik és egyben legeredményesebb próba tanulási görbéje, a harmadik fázisban.

Összességében a hiperparaméter-optimalizálás eredményei azt mutatják, hogy a megfelelően regularizált, közepes méretű LSTM-modellek képesek a legjobb általánosítási teljesítményt nyújtani. A túl nagy modellek nem eredményeztek arányos teljesítménynövekedést, miközben jelentősen növelték a tanítási költséget és a számítási erőforrásigényt.

Áttérve a növekményes tanítóhalmaz kiértékelésére, láthatóvá válik, hogy az ingadozás mértéke jelentősen csökken. A legnagyobb negatív eltérés alig közelíti meg az ötszázados eltérést, és ez a tanítóhalmaz növekedésével egyre kisebb mértékű lesz, miközben az F1-érték közelít a 0.98 értékhez. Érdekes megfigyelés, hogy az 8-as adag, amely 13911 üzenetet tartalmaz, hozott a legjobb eredményt, és az utána lévő tanítóhalmaz növelése rosszabb eredményeket hozott. A 5.7 ábráról nehezen olvasható le, azonban a JSON-fájlokban tárolt pontos értékekből látható, hogy a 10-es adag tanítóhalmaz esetében az F1-érték több mint egy századponttal rosszabb eredményt produkált, mint az 8-as.



5.7. ábra: A harmadik fázis növekményes tanulási görbéje. A variancia csökkenése és stabilizálódása érzékelhető.

Ez nem feltétlenül az általánosító képesség romlására utal, hanem több tényező együttes hatásának következménye is lehet. Lehetséges például, hogy az újonnan bevont üzenetek nagyobb zajt vagy nehezebben általánosítható mintázatokat vezettek be a tanítási folyamatba. A természetes nyelvi adathalmazok esetében gyakori, hogy a tanítóadat bővítésével nem kizárólag hasznos információ kerül a modellhez, hanem redundáns, bizonytalan vagy akár félrevezető minták is, amelyek átmenetileg ronthatják az optimalizáció stabilitását.

Emellett az is elképzelhető, hogy a rögzített hiperparaméterek az adott tanítóhalmaz méretéhez közel optimálisnak bizonyultak, azonban a nagyobb adatmennyiség már eltérő tanulási dinamikát igényelt volna. Ilyen esetben például a tanulási ráta, a regularizáció mértéke vagy az epochok száma már nem biztosít megfelelő konvergenciát. A mély neurális hálók optimalizációja bonyolult optimalizációs probléma, ezért kisebb teljesítményingadozások még stabil tanítás mellett is természetesnek tekinthetők [3].

Az eredmények ugyanakkor pozitív tendenciát mutatnak. A tanítóhalmaz növekedésével az F1-érték varianciája fokozatosan csökkent, a görbe stabilabbá vált, és a teljesítmény tartósan magas szinten maradt. Ez arra utal, hogy a modell általánosító képessége összességében javult, még akkor is, ha egyes részhalmazok esetében kisebb lokális teljesítménycsökkenés volt megfigyelhető.

6. fejezet

Összefoglalás és továbbhaladási irány

6.1. Összefoglalás (magyar)

A dolgozat a kéretlen (spam) és valós (ham) elektronikus levelek automatikus megkülönböztetésének vizsgálatával foglalkozik mélytanulási módszerek alkalmazásával a természetes nyelvfeldolgozás területén. A probléma napjainkban is kiemelten releváns, mivel a nagy mennyiségű szöveges kommunikáció automatikus szűrése számos informatikai rendszer alapvető feladata. A kéretlen levélszemét felismerés nehézségét az adja, hogy az osztályozás erősen függ a nyelvi mintázatoktól, az adatok eloszlásától, valamint a tanítóhalmaz összetételétől. A dolgozat célja egy olyan bináris szövegosztályozó modell megtervezése, betanítása és kiértékelése volt, amely kiegyensúlyozott teljesítményt nyújt, különösen az F1-érték szempontjából.

A feldolgozási lánc első lépését az adatok előfeldolgozása és tokenizálása jelentette, amely SentencePiece alapú tokenizáló segítségével történt. A modellek változó hosszúságú szekvenciákon tanultak, szükség esetén kitöltéssel (padding) egységesítve a bemenetet. Az alkalmazott architektúra kétirányú hosszú-rövid távú memóriával rendelkező neurális hálózatra (BiLSTM) épült, amely beágyazási rétegből, több LSTM-rétegből és egy sűrű osztályozó fejből állt, dropout regularizáció alkalmazásával. A tanítás PyTorch környezetben valósult meg. A hiperparaméter-optimalizálás random kereséssel történt, több egymásra épülő fázisban. A kezdeti széles keresési tér fokozatos szűkítésével lehetővé vált a stabilabb és számítási szempontból hatékonyabb konfigurációk feltárása. A vizsgált hiperparaméterek közé tartozott többek között a tanulási ráta, a batch méret, a maximális szekvenciahossz, a rejtett rétegek mérete és száma, a dropout mértéke, a sűrű réteg mérete, a gradiensnorma korlátozása, valamint az epochok száma. A modellek teljesítményének értékelése elsősorban az F1-érték alapján történt, amelyet precizitás, visszahívás, konfúziós mátrixok és tanulási görbék elemzése egészített ki. Ez különösen fontosnak bizonyult azokban az esetekben, amikor a pontosság önmagában félrevezető képet adott a modell viselkedéséről.

Az eredmények alapján megfelelő hiperparaméterek mellett a kétirányú hosszú-rövid memória modellek stabilan magas, körülbelül 0.97-0.98 közötti F1-értéket tudtak elérni. A hiperparaméter-tér fokozatos szűkítése jelentősen csökkentette az instabil vagy aránytalanul erőforrás-igényes konfigurációk számát. A növekményes tanítóhalmazos kísérletek alapján megfigyelhető volt, hogy a tanítóadat mennyiségének növelése általában javította vagy stabilizálta a teljesítményt, azonban a javulás nem minden esetben volt monoton. Egyes részhalmazok átmeneti teljesítményromlást okoztak, amely valószínűleg zajosabb vagy nehezebben általánosítható minták megjelenésével magya-

rázható.

A dolgozat egyik legfontosabb megállapítása, hogy a kétirányú hosszú-rövid memória modellek erősen függenek a hiperparaméterek megválasztásától. A legjobb eredményeket általában a kisebb vagy közepes méretű, stabilabban tanítható modellek érték el, míg a nagyobb paraméterszámú architektúrák nem eredményeztek arányos teljesítménynövekedést, vagy instabilitáshoz vezettek. Emellett a hiperparaméter-tér fokozatos és tudatos szűkítése gyakorlati szempontból is meghatározó tényezőnek bizonyult, mivel a stabil és reprodukálható futások hiányában a modellek összehasonlítása és a tanulási görbék értelmezése is jelentősen nehezebbé válik.

A kísérletek egyetlen laptop GPU használatával kerültek végrehajtásra, részletes futásidő- és erőforrás-mérés nélkül. Emiatt a számítási költségek csak tapasztalati alapon voltak becsülhetők. További korlátot jelentett, hogy a tanítóhalmaz részhalmozainak pontos összetétele nem került rögzítésre, így a tanítóhalmazban lévő üzenetek vizsgálata ellehetetlenült.

A jövőbeni fejlesztések egyik fontos iránya lehet a tanítási folyamat erőforrásigényének részletesebb vizsgálata, például a futási idő, a memóriahasználat és az egyes modellek számítási költségének összehasonlításával. Emellett érdemes lenne a tanító-, validációs- és teszhalmazok pontosabb nyilvántartását is megvalósítani, hogy a későbbi kísérletek könnyebben visszakövethetők és reprodukálhatók legyenek. További lehetőséget jelenthet modern, transzformer-alapú nyelvi modellek vagy előtanított modellek vizsgálata hasonló szövegosztályozási feladatokon. A bemutatott módszerek nemcsak elektronikus levelek elemzésére alkalmazhatók, hanem más szöveges adatok vizsgálatára is, például rosszindulatú programkódok vagy konfigurációs fájlok felismerésére. Ilyen esetekben a nagy nyelvi modellek elsősorban támogató szerepet tölthetnek be, segítve az emberi szakértők munkáját a gyanús minták azonosításában és elemzésében.

6.2. Summary (english)

This thesis investigated the automatic classification of spam and legitimate (ham) e-mails using deep learning methods. A Bidirectional Long Short-Term Memory (BiLSTM) architecture was trained and evaluated on tokenized text data in order to analyze how different hyperparameter settings influence classification performance and model stability. The experiments focused not only on maximizing performance, but also on understanding the relationship between training behavior, generalization capability, and computational cost.

Hyperparameter optimization was performed in multiple phases using random search with gradually reduced search spaces. The evaluated parameters included learning rate, batch size, sequence length, hidden layer size, dropout rate, gradient clipping threshold, and the number of training epochs. Model performance was primarily measured using the F1-score, supported by precision, recall, confusion matrices, and learning curves.

The results showed that properly configured BiLSTM models were capable of achieving stable F1-scores around 0.97–0.98. The experiments also demonstrated that larger models with higher parameter counts did not necessarily improve performance and often introduced less stable training behavior. In many cases, smaller or medium-sized models with stronger regularization provided better generalization and more reliable convergence. The cumulative training set experiments further showed that increasing the amount of training data generally stabilized the learning process, although certain

data subsets temporarily reduced performance due to noisier or more difficult samples.

Overall, the thesis highlights the importance of systematic hyperparameter optimization and detailed evaluation beyond simple accuracy metrics. The results confirm that stable and well-regularized BiLSTM models can provide strong performance for spam classification tasks while remaining computationally manageable.

Future work may include a more detailed analysis of computational efficiency, improved experiment reproducibility, and the evaluation of transformer-based or pretrained language models on similar text classification problems.

Források

- [1] James Bergstra és Yoshua Bengio. „Random search for hyper-parameter optimization”. *J. Mach. Learn. Res.* 13.null (2012. febr.), 281–305. ISSN: 1532-4435.
- [2] Liriam Enamoto és tsai. „Multi-label legal text classification with BiLSTM and attention”. *International Journal of Computer Applications in Technology* 68 (2022. jan.), 369. old. DOI: 10.1504/IJCAT.2022.125186.
- [3] Ian Goodfellow, Yoshua Bengio és Aaron Courville. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.
- [4] IBM. *Artificial intelligence*. URL: <https://www.ibm.com/think/topics/artificial-intelligence> (elérés dátuma 2026. 04. 29.).
- [5] IBM. *What is a confusion matrix?* International Business Machines Corporation (IBM). URL: <https://www.ibm.com/think/topics/confusion-matrix> (elérés dátuma 2026. 05. 11.).
- [6] Kamran Kowsari és tsai. „Text classification algorithms: A survey”. *Information* 10.4 (2019), 150. old. DOI: 10.3390/info10040150. URL: <https://doi.org/10.3390/info10040150>.
- [7] Taku Kudo és John Richardson. „SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing”. *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Szerk. Eduardo Blanco és Wei Lu. Brussels, Belgium: Association for Computational Linguistics, 2018. nov., 66–71. old. DOI: 10.18653/v1/D18-2012.
- [8] Qian Li és tsai. „A Survey on Text Classification: From Traditional to Deep Learning”. *ACM Transactions on Intelligent Systems and Technology* 13.2 (2022), 1–39. old. DOI: 10.1145/3495162.
- [9] Gang Liu és Jiabao Guo. „Bidirectional LSTM with attention mechanism and convolutional layer for text classification”. *Neurocomputing* 337 (2019), 325–338. old. ISSN: 0925-2312. DOI: <https://doi.org/10.1016/j.neucom.2019.01.078>. URL: <https://www.sciencedirect.com/science/article/pii/S0925231219301067>.
- [10] Vangelis Metsis, Ion Androutsopoulos és Georgios Paliouras. „Spam Filtering with Naive Bayes - Which Naive Bayes?": 2006. jan.
- [11] Steven T. Piantadosi. „Zipf’s word frequency law in natural language: A critical review and future directions”. *Psychonomic Bulletin & Review* 21.5 (2014), 1112–1130. old.

- [12] Procogia. *The Interquartile Range Method For Outliers*. URL: <https://procogia.com/interquartile-range-method-for-reliable-data-analysis/> (elérés dátuma 2026. 05. 04.).
- [13] Hao Qin. „BiLSTM Text Classification Incorporating Attentional Mechanisms”. *Academic Journal of Computing & Information Science* 6.13 (2023), 92–98. old. DOI: 10.25236/AJCIS.2023.061314. URL: <https://francispress.com/uploads/papers/louD4xcIlZAI6NdXM0126npbxAqYc0cxIOMK41gW.pdf>.
- [14] Cole Stryker és Jim Holdsworth. *What is NLP?* International Business Machines Corporation (IBM). URL: <https://www.ibm.com/think/topics/natural-language-processing> (elérés dátuma 2026. 04. 29.).
- [15] Kamal Taha és tsai. „A comprehensive survey of text classification techniques and their research applications: Observational and experimental insights”. *Computer Science Review* 54 (2024), 100664. old. DOI: 10.1016/j.cosrev.2024.100664. URL: <https://doi.org/10.1016/j.cosrev.2024.100664>.
- [16] U.S. Census Bureau. *Data science*. URL: <https://www.census.gov/topics/research/data-science.html> (elérés dátuma 2026. 04. 29.).
- [17] Mengnan Wang. „Research on Text Classification Method Based on NLP”. *Advances in Computer, Signals and Systems* 7 (2023), 93–100. old. DOI: 10.23977/acss.2023.070213.
- [18] M. Wiechmann és M. Noppel. *Enron Spam Dataset*. Data set. GitHub. URL: https://github.com/MWiechmann/enron_spam_data (elérés dátuma 2026. 05. 06.).
- [19] Peng Zhou és tsai. *Text Classification Improved by Integrating Bidirectional LSTM with Two-dimensional Max Pooling*. 2016. arXiv: 1611.06639 [cs.CL]. URL: <https://arxiv.org/abs/1611.06639>.

Repository használati útmutató

Ez az útmutató a tároló (GitHub / mellékelt adathordozó) tartalmának áttekintéséhez és futtatásához szükséges lépéseket foglalja össze.

Beállítás

1. CUDA-képes videokártya szükséges a futtatáshoz.
2. Python 3.10.0 telepítése.
3. Csomagok telepítése: `pip install -r requirements.txt`

Használat

Adathalmaz feldolgozása

A `notebooks/` mappában található, Jupyter-notebookokat sorban le kell futtatni.

Model

A tároló gyökerében a `main.py` a belépési pont.

Hiperparaméter-keresés

```
python main.py tune [--num-trials 20]
```

A próbák száma opcionális. A reprodukálhatóság érdekében az alapértelmezett érték **20**, ezzel lett mindhárom fázis futtatva.

Növekményes tanítóhalmazon tanulási görbe

```
python main.py learning-curve --num-portions 10
```

A részek száma szintén opcionális. Az alapértelmezés **10**. Ezzel lett kiértékelve a növekményes adathalmazokon tanított modellek **általánosító** képessége.

Jegyzékfa

Az alábbi fa segít eligazodni a tárolóba. A `results` jegyzék tartalmazza a futási eredményeket, így ennek megtekintésével futtatás nélkül is tanulmányozhatóak az eredmények.

```
uni-thesis/
|
+-- main.py                # Belépési pont: tune vagy learning-curve
+-- requirements.txt        # pip-függőségek
+-- README.md
+-- .gitignore              # Nem verziókezelt fájlok listája
|
+-- notebooks/
|   +-- 1-eda-1.ipynb       # EDA
|   +-- 2-data-cleaning.ipynb
```

```

|   +-- 3-verification.ipynb
|   +-- 4-tokenization.ipynb # Tokenizáló
|   +-- 5-validation.ipynb   # Validációs lépések
|   '-- (*.png)              # Futtatáskor generált ábrák
|
+-- data/
|   +-- corpora/
|   |   +-- raw/              # Enron, CSV, train/teszt/validáció
|   |   '-- processed/       # Előfeldolgozott és tokenizer tanító szöveg
|   '-- models/
|       +-- lstm_tokenizer.model
|       +-- train_ids.pkl, train_pieces.pkl
|       +-- learning_curve/
|       |   '-- portion_01..10/
|       |   |       +-- best_model.pt
|       |   |       +-- checkpoint_epoch*.pt
|       |   |       +-- eval_test_metrics.json
|       |   |       +-- test_confusion_matrix.png
|       |   '-- training_curves.png
|       '-- tuning/
|           '-- trial_001 .. trial_XXX/
|           +-- best_model.pt
|           +-- checkpoint_epoch*.pt
|           +-- eval_test_metrics.json
|           +-- test_confusion_matrix.png
|           '-- training_curves.png
|
+-- src/
|   +-- config/               # config.json, hiperparaméterek
|   +-- data/
|   +-- models/              # bilstm.py
|   +-- training/
|   +-- evaluation/
|   +-- experiments/         # tuning.py, learning_curve.py
|   '-- infrastructure/
|
+-- thesis/                  # Szakdolgozat LaTeX
|
'-- results/
    +-- preprocessing/       # Notebookok kimenete (Markdown, ábrák)
    +-- experiements/        # Modellek kiértékelései (archívum)
    |   +-- 1_phase_one_tuning/
    |   +-- 2_phase_two_learning_curve/
    |   +-- 2_phase_two_tuning/
    |   +-- 3_phase_three_learning_curve/
    |   '-- 3_phase_three_tuning/
    '-- SPAM_HAM_...pdf      # Dolgozat PDF (ha ide kerül)

```